



KLE Technological University

Creating Value,
Leveraging Knowledge

A Minor Project report on

An End-to-End Ambient AI System for Clinical Documentation using Automatic Speech
Recognition and Large Language Models

Submitted

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

IN

Department of Computer Science & Engineering

Submitted By

Name	USN
Pavan Nagesh	01FE24BCS402
Sanjana Shetti	01FE23BCS057
Shrutika Ambure	01FE23BCS198
Rohan Kabadi	01FE23BCS418

Under the guidance of

Prof. Vijay Biradar

Professor

Department of Computer Science and Engineering

KLE Technological University, Hubballi

2025-2026



KLE Technological University

Creating Value,
Leveraging Knowledge

Department of Computer Science and Engineering

2025-2026

CERTIFICATE

This is to certify that the project entitled “An End-to-End Ambient AI System for Clinical Documentation using Automatic Speech Recognition and Large Language Models” is a bonafide work carried out by the student team Pavan Nagesh (01FE24BCS402), Sanjana Shetti (01FE23BCS057), Shrutika Ambure (01FE23BCS198), and Rohan Kabadi (01FE23BCS418), in partial fulfillment of the completion of 6th semester Minor Project during the academic year 2025–2026.

The project focuses on the development of an AI-driven clinical documentation system capable of converting doctor–patient conversations into structured medical records through Automatic Speech Recognition, speaker diarization, Large Language Models, SOAP note generation, and Hybrid Retrieval-Augmented Generation (RAG) based continuity summarization.

The project report has been approved as it satisfies the academic requirements prescribed for the above said course.

Guide Name

Prof. Vijay Biradar

Head, CSE

Dr. Vijayalakshmi M.

External Viva-Voce

Name of the examiners

Signature with date

1 _____

2 _____

ABSTRACT

Clinical documentation is an essential yet time-consuming component of healthcare workflows. Doctors often spend a considerable amount of time manually recording patient interactions, reducing the time available for direct patient care. Existing speech-to-text systems primarily focus on transcription and do not provide structured clinical understanding, contextual continuity, or intelligent summarization of medical conversations.

This project presents an end-to-end Ambient AI system for automated clinical documentation using Automatic Speech Recognition (ASR), speaker diarization, Large Language Models (LLMs), and Hybrid Retrieval-Augmented Generation (RAG). The system captures doctor-patient consultation audio and processes it through a multi-stage AI pipeline to generate structured medical outputs. The implemented pipeline includes audio ingestion, speech transcription, speaker role identification, clinical entity extraction, SOAP note generation, and automated Personally Identifiable Information (PII) redaction.

The proposed system additionally incorporates a Hybrid RAG-based continuity framework that combines SQL-based recency retrieval with vector-based semantic retrieval to generate clinically relevant summaries from historical patient consultations. Semantic embeddings are generated using sentence-transformer models and ranked using cosine similarity to retrieve contextually relevant medical information.

The backend of the system is implemented using FastAPI and PostgreSQL, while the frontend is developed using React and TypeScript. SpeechBrain ECAPA-TDNN is utilized for voice embedding generation, and Sarvam Saaras v3 is used for multilingual ASR and diarization tasks.

The developed system reduces documentation overhead, improves clinical record organization, enhances consultation continuity, and provides a scalable foundation for future intelligent healthcare documentation systems.

Keywords : *Ambient AI, Clinical Documentation, Automatic Speech Recognition, Speaker Diarization, Large Language Models, SOAP Notes, Hybrid RAG, FastAPI, Semantic Retrieval, Healthcare AI.*

ACKNOWLEDGEMENT

We express our sincere gratitude to the management and faculty members of KLE Technological University for providing us with the opportunity and support required for the successful completion of this minor project work. We would like to thank Dr. Ashok Shettar, Pro-Chancellor, Dr. P. G. Tewari, Vice-Chancellor, and Dr. B. S. Anami, Registrar, for their vision, encouragement, and institutional support.

We also express our gratitude to Dr. Meena S. M., Professor and Dean of the School of Computer Science and Engineering, and Dr. Vijayalakshmi M., Professor and Head of the Department of Computer Science and Engineering, for providing academic direction and support throughout the course of the project.

We sincerely thank our project guide, Prof. Vijay Biradar, Department of Computer Science and Engineering, for his valuable guidance, continuous encouragement, technical suggestions, and constant support during the development of this project.

We would also like to acknowledge our industry mentor, Mr. Shankaran Sitaraman, CEO of Traction Layer AI, for giving us the opportunity to work on a real-world healthcare AI problem statement and for sharing valuable industry insights during the implementation of the project.

Pavan Nagesh - 01FE24BCS402
Sanjana Shetti - 01FE23BCS057
Shrutika Ambure - 01FE23BCS198
Rohan Kabadi - 01FE23BCS418

CONTENTS

ABSTRACT	i
CONTENTS	iii
LIST OF TABLES	iv
LIST OF FIGURES	vi
1 INTRODUCTION	1
1.1 Preamble (Provide Introduction of the project)	1
1.2 Motivation	2
1.3 Literature Study	2
1.4 Problem Definition	3
1.5 Objectives of the project	3
2 SOFTWARE REQUIREMENT SPECIFICATION	5
2.1 Overview of SRS	5
2.2 Requirement Specifications	6
2.2.1 Functional Requirements	6
2.2.2 Use Case Diagrams	6
2.2.3 Use Case Descriptions Using Scenarios	7
2.2.4 Nonfunctional Requirements	8
2.3 Software and Hardware Requirement Specifications	9
3 PROPOSED SYSTEM	10
3.1 Description of Proposed System	10
3.2 Description of Target Users	12
3.3 Advantages and Applications of Proposed System	12
3.4 Scope (Boundary of Proposed System)	13
3.5 System Workflow	14
4 SYSTEM DESIGN	16
4.1 Architecture of the system	16
4.2 Class Diagram (with brief explanation)	18
4.3 Sequence diagram	19
4.4 ER diagram and schema	20

4.5	State transition diagram	21
4.6	Data Set Description	22
5	IMPLEMENTATION	23
5.1	Proposed Methodology	23
5.2	Description of Modules	24
6	TESTING	28
6.1	Test Plan and Test Cases	28
6.2	Acceptance Test Plan and Test Cases	29
6.3	Unit Test Plan and Test Cases	30
6.4	Integration Testing	31
6.5	Performance Testing	32
7	RESULTS & DISCUSSIONS	33
7.1	Doctor Voice Enrollment Results	33
7.2	Dashboard and Consultation Interface Results	35
7.3	Speaker Diarization Results	36
7.4	SOAP Note Generation Results	37
7.5	Hybrid RAG Retrieval Results	38
7.6	Performance Analysis	39
7.7	Discussion	39
8	CONCLUSIONS AND FUTURE SCOPE	41
8.1	Conclusion	41
8.2	Future Scope	42
	REFERENCES	43
	Appendix A APPENDICES	44
A.1	Screen Shots	44
A.2	Gantt Chart	47
A.3	Glossary	48
A.4	Description of Tools and Technology Used	48
A.5	Blue Print	49

LIST OF TABLES

6.1	Acceptance Test Cases	29
6.2	Unit Test Cases	30
6.3	Integration Testing Results	31
6.4	Performance Testing Results	32
7.1	Performance Analysis Results	39

LIST OF FIGURES

2.1	Use Case Diagram of Ambient AI System	7
3.1	Overall Architecture of the Ambient AI Clinical Documentation System	11
3.2	Advantages and Applications of the Proposed System	13
3.3	System Workflow of the Proposed Ambient AI Platform	14
4.1	System Architecture of the Ambient AI Platform	17
4.2	Class Diagram of Ambient AI System	18
4.3	Sequence Diagram for Consultation Processing	19
4.4	Entity Relationship Diagram of Ambient AI Database	20
4.5	State Transition Diagram of Consultation Processing	21
5.1	Implementation Workflow of Ambient AI System	24
5.2	Module Interaction Workflow	27
7.1	Doctor Language Selection During Voice Enrollment	34
7.2	Doctor Voice Recording and Verification Interface	34
7.3	Frontend Dashboard and Consultation Interface	35
7.4	Speaker Diarization Result	36
7.5	Generated SOAP Notes	37
7.6	Hybrid RAG Retrieval Result	38
A.1	Doctor Language Selection Interface	44
A.2	Doctor Voice Enrollment Interface	45
A.3	Clinician Dashboard Interface	45
A.4	Speaker Diarization Output	46
A.5	Generated SOAP Notes	46
A.6	Hybrid RAG Retrieval Output	47
A.7	Gantt Chart	47
A.8	Overall System Architecture	50
A.9	Clinical Consultation Workflow Diagram	50
A.10	State Transition Diagram	51
A.11	Sequence Diagram	51
A.12	Entity Relationship Diagram	52
A.13	Class Diagram	52
A.14	Module Interaction Workflow	53

A.15 Hybrid RAG Retrieval and System Workflow	53
---	----

Chapter

1

INTRODUCTION

Artificial Intelligence has significantly transformed several industries by enabling intelligent automation and data-driven decision-making. In the healthcare domain, one of the major administrative challenges faced by clinicians is the time spent on manual clinical documentation. Doctors are often required to record patient interactions, prepare consultation notes, summarize diagnoses, and maintain patient history after every consultation session. This process is repetitive, time-consuming, and prone to inconsistencies, especially during high patient workloads.

The proposed project, titled “*An End-to-End Ambient AI System for Clinical Documentation using Automatic Speech Recognition and Large Language Models*”, aims to automate the process of generating structured medical records directly from doctor–patient conversations. The system utilizes Automatic Speech Recognition (ASR), speaker diarization, Large Language Models (LLMs), and Hybrid Retrieval-Augmented Generation (RAG) to convert raw consultation audio into clinically meaningful documentation [1, 2, 3].

The developed system processes consultation audio through a Directed Acyclic Graph (DAG)-based AI pipeline consisting of transcription, speaker role identification, clinical entity extraction, SOAP note generation, and privacy-preserving redaction. Additionally, the system incorporates a continuity-aware retrieval mechanism capable of generating semantically relevant summaries from historical consultations using vector-based semantic search combined with SQL-based recency retrieval [3, 4].

1.1 Preamble (Provide Introduction of the project)

Healthcare professionals spend a considerable amount of time maintaining patient records and preparing consultation summaries. Traditional documentation systems primarily depend on manual data entry, which increases administrative burden and reduces the amount of time available for direct patient care. Although modern speech-to-text systems assist in transcription, they are generally limited to plain text generation and do not provide structured clinical understanding.

The proposed Ambient AI system addresses this challenge by integrating multiple AI technologies into a unified clinical documentation pipeline. The system captures doctor–patient conversations, performs multilingual speech recognition, identifies speaker roles, extracts clin-

ical entities such as symptoms and medications, and generates structured SOAP notes that can be reviewed and edited by clinicians before storage [1, 5].

The project also focuses on ensuring continuity of care by introducing a Hybrid RAG-based retrieval system capable of retrieving semantically relevant historical consultation information. This allows doctors to quickly understand previous medical context during follow-up consultations [3].

1.2 Motivation

The motivation behind this project arises from the growing need for intelligent healthcare documentation systems capable of reducing clinician workload while improving the quality and consistency of patient records. In many healthcare environments, clinicians spend a substantial portion of their time writing consultation summaries instead of directly interacting with patients.

Manual documentation is not only time-consuming but also susceptible to incomplete recording of important medical details. Furthermore, retrieving relevant historical consultation information during follow-up visits is often difficult due to the large volume of unstructured patient records.

Recent advancements in Artificial Intelligence, speech processing, and Large Language Models have created opportunities to automate several healthcare documentation tasks. This project was motivated by the idea of combining speech recognition, semantic understanding, and intelligent retrieval into a single integrated system capable of generating clinically useful documentation in an efficient and scalable manner [2, 6].

1.3 Literature Study

Several existing healthcare documentation systems focus primarily on speech transcription and lack advanced semantic understanding of clinical conversations. Conventional ASR systems convert speech into text but do not identify speaker roles or generate structured medical summaries. Additionally, many systems fail to maintain contextual continuity across multiple patient visits.

Recent research in healthcare AI has explored the use of Large Language Models for medical summarization and clinical note generation. LLMs have shown promising results in extracting medical entities, generating SOAP notes, and improving natural language understanding within healthcare workflows [6, 2]. However, many existing implementations rely entirely on cloud-based APIs and often lack retrieval mechanisms capable of incorporating historical patient context.

Retrieval-Augmented Generation (RAG) has emerged as an effective technique for combining semantic retrieval with language generation. Hybrid retrieval methods that combine vector-based semantic search with structured database retrieval have demonstrated improved contextual relevance in information retrieval systems [3, 7].

The proposed system extends these concepts by integrating ASR, speaker diarization, LLM-based structuring, SOAP note generation, and Hybrid RAG-based continuity summarization into a single end-to-end clinical documentation pipeline.

1.4 Problem Definition

Clinical documentation remains one of the major administrative challenges in modern healthcare systems. Doctors are required to manually record patient interactions, prepare structured consultation summaries, and maintain continuity across multiple patient visits. Existing documentation workflows consume significant time and may lead to inconsistencies, incomplete records, and clinician fatigue.

Traditional speech-to-text solutions are insufficient because they mainly provide raw transcription without understanding the semantic context of the conversation. They do not identify doctor and patient speakers, extract structured medical information, or generate standardized clinical notes. Furthermore, retrieving clinically relevant historical consultation information during follow-up visits remains difficult in conventional systems.

Therefore, there is a need for an intelligent and scalable system capable of automatically converting doctor–patient conversations into structured, privacy-compliant clinical records while also supporting semantically aware retrieval of historical consultation data [1, 3].

1.5 Objectives of the project

The primary objective of the proposed project is to design and develop an AI-driven healthcare documentation system capable of automating clinical record generation from doctor–patient conversations.

The specific objectives of the project are as follows:

- To develop an end-to-end AI pipeline for automated clinical documentation.
- To implement Automatic Speech Recognition and speaker diarization for consultation audio processing.
- To identify doctor and patient speaker roles using semantic analysis.
- To extract structured clinical entities such as symptoms, medications, vitals, and diagnoses.

- To generate standardized SOAP notes using Large Language Models.
- To implement automated Personally Identifiable Information (PII) redaction for privacy preservation.
- To design a Hybrid RAG-based continuity system combining SQL retrieval and semantic vector search.
- To develop a user-friendly frontend interface for consultation review, editing, and patient history management.

Chapter 2

SOFTWARE REQUIREMENT SPECIFICATION

The Software Requirement Specification (SRS) document defines the functional and non-functional requirements of the proposed Ambient AI based clinical documentation system. It provides a detailed description of the system functionalities, performance expectations, usability considerations, security requirements, and hardware/software dependencies necessary for successful implementation [8].

The proposed system is designed to automate the generation of structured clinical documentation from doctor–patient conversations using Artificial Intelligence techniques such as Automatic Speech Recognition (ASR), speaker diarization, Large Language Models (LLMs), and Hybrid Retrieval-Augmented Generation (RAG) [1, 2, 3]. The SRS document serves as a foundation for system design, development, testing, deployment, and maintenance.

2.1 Overview of SRS

The Software Requirement Specification describes the operational behavior and expected functionality of the proposed system. The system accepts consultation audio as input and processes it through multiple AI-driven stages to generate structured medical documentation.

The SRS document defines:

- Functional capabilities of the system
- User interactions and workflows
- Performance and security constraints
- Software and hardware dependencies
- System usability and scalability requirements

The document also acts as a reference for developers, testers, reviewers, and project stakeholders during different phases of the project lifecycle.

2.2 Requirement Specifications

Requirement specifications describe the expected behavior and operational characteristics of the proposed system. Functional requirements define what the system should perform, while non-functional requirements define the quality constraints under which the system operates [8].

The proposed Ambient AI system includes functionalities related to consultation audio processing, speaker role identification, clinical entity extraction, SOAP note generation, semantic retrieval, patient record management, and privacy-preserving data handling [5, 3].

2.2.1 Functional Requirements

- **FR1:** The system shall allow clinicians to upload or record consultation audio through the frontend interface.
- **FR2:** The system shall perform Automatic Speech Recognition and speaker diarization on uploaded consultation audio [1].
- **FR3:** The system shall identify doctor and patient speaker roles using semantic analysis.
- **FR4:** The system shall extract structured clinical entities such as symptoms, medications, conditions, and vitals from transcripts.
- **FR5:** The system shall generate editable SOAP notes for every consultation [5].
- **FR6:** The system shall perform automated PII redaction before storing consultation records.
- **FR7:** The system shall store patient consultations and maintain historical records.
- **FR8:** The system shall implement Hybrid RAG-based continuity summarization using SQL retrieval and semantic vector search [3, 7, 4].
- **FR9:** The system shall provide authentication and authorization for clinicians.
- **FR10:** The system shall allow clinicians to review, edit, and save generated documentation.

2.2.2 Use Case Diagrams

The use case diagram illustrates the interaction between the clinician and the Ambient AI system. Major use cases include clinician authentication, audio upload, consultation processing, SOAP generation, patient history retrieval, and continuity summary generation.

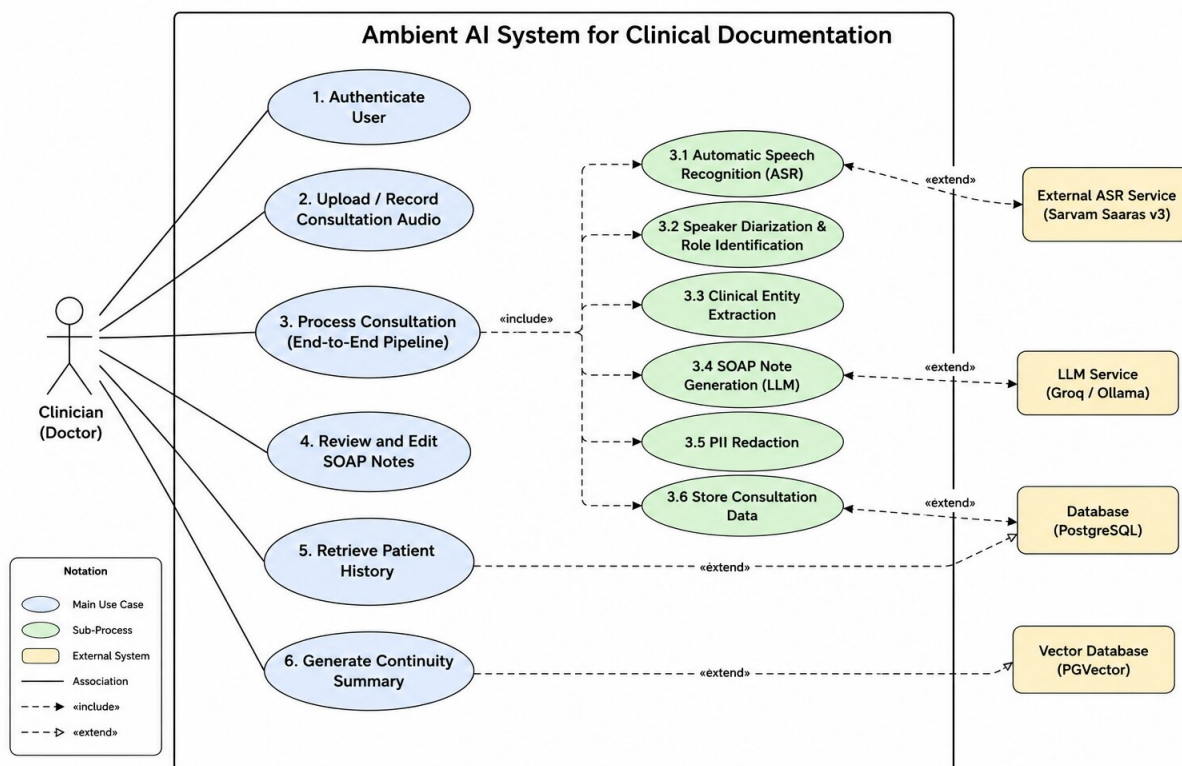


Figure 2.1: Use Case Diagram of Ambient AI System

2.2.3 Use Case Descriptions Using Scenarios

Use Case: Consultation Processing

- **Actor:** Clinician
- **Precondition:** User must be authenticated into the system.
- **Input:** Consultation audio recording or uploaded audio file.
- **Process:**
 - Audio is uploaded to the backend.
 - ASR and diarization are performed.
 - Speaker roles are identified.
 - Clinical entities are extracted.
 - SOAP notes are generated and redacted.
 - Consultation data is stored in the database.
- **Output:** Structured SOAP notes and consultation summary.

- **Postcondition:** Consultation history becomes available for continuity retrieval.

2.2.4 Nonfunctional Requirements

- **NFR1:** The system should support scalable consultation processing for multiple users.
- **NFR2:** The system should provide secure authentication and protected access to patient records.
- **NFR3:** The system should provide low-latency response during AI pipeline execution.
- **NFR4:** The system should ensure data privacy and confidentiality.
- **NFR5:** The frontend interface should be user-friendly and responsive.

Performance Requirements

The system should efficiently process consultation audio with minimal delay. Parallel execution is utilized during AI processing stages to reduce latency. The Hybrid RAG retrieval system should provide continuity summaries within acceptable response time limits for practical clinical usage [3].

Reliability and Scalability

The system should maintain reliability, scalability, usability, maintainability, and fault tolerance throughout execution. Failures occurring during embedding generation or LLM parsing should not interrupt the overall consultation processing workflow.

Security Requirements

The system implements JWT-based authentication, password hashing using bcrypt, OTP verification, and role-based access validation. Personally Identifiable Information (PII) is redacted before persistence to ensure privacy protection and secure clinical record management [9, 10].

Usability

The frontend interface is designed to provide a simple and intuitive workflow for clinicians. The system supports audio upload, live recording, consultation review, SOAP editing, and patient history access through a responsive user interface.

Extensibility

The system architecture is modular and extensible, allowing independent upgrades of AI models, retrieval mechanisms, frontend components, and backend services without affecting the entire pipeline.

2.3 Software and Hardware Requirement Specifications

The proposed Ambient AI system requires both software and hardware components for development, deployment, and execution. The backend services, AI models, frontend application, and database components are integrated to support end-to-end clinical documentation processing.

- **Operating System:** Windows 10/11 or Linux
- **Frontend Technologies:** React.js, TypeScript, Tailwind CSS
- **Backend Technologies:** FastAPI, Python, SQLAlchemy [11]
- **Database:** PostgreSQL [12]
- **AI Frameworks:** SpeechBrain, sentence-transformers, NumPy [?]
- **ASR Service:** Sarvam Saaras v3
- **LLM Services:** Groq API and Ollama
- **Processor:** Intel i5/Ryzen 5 or higher
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 20 GB free storage
- **Internet Connectivity:** Required for cloud-based inference APIs

““

Chapter 3

PROPOSED SYSTEM

The proposed system is an AI-driven clinical documentation platform developed to automate the generation of structured medical records from doctor–patient conversations. The system combines Automatic Speech Recognition (ASR), speaker diarization, semantic analysis, Large Language Models (LLMs), and Hybrid Retrieval-Augmented Generation (RAG) within a single workflow pipeline [1, 2, 3].

In many healthcare environments, clinicians spend considerable time manually preparing consultation notes after every patient interaction. This often increases administrative workload and reduces the amount of time available for direct patient care. The developed system attempts to address this issue by automatically converting consultation audio into structured SOAP notes and continuity-aware summaries.

The architecture of the platform follows a modular design approach so that frontend interfaces, backend APIs, AI models, and retrieval services can function independently while remaining connected within the overall workflow. This modularity also simplified debugging and integration during implementation.

3.1 Description of Proposed System

The Ambient AI platform processes consultation audio through multiple AI-based stages in order to generate clinically meaningful documentation. The overall system consists of frontend interfaces, backend APIs, AI processing services, retrieval modules, and database storage components.

The workflow begins when a clinician logs into the platform and uploads or records consultation audio. The uploaded audio is then processed using ASR and speaker diarization models to generate textual transcripts and identify separate speaker segments [1]. During implementation, handling overlapping speech and inconsistent audio quality required additional preprocessing to improve transcription clarity.

After transcription, the generated text is processed using Large Language Models to identify doctor and patient roles, extract important clinical entities, and generate structured SOAP notes [2, 6, 5]. Personally Identifiable Information (PII) is redacted before storage in order to maintain patient privacy and confidentiality.

To improve continuity during follow-up consultations, the system also implements a Hy-

brid RAG-based retrieval mechanism. This module combines SQL-based recency retrieval with semantic vector similarity search to retrieve contextually relevant historical consultation information [3, 7, 4]. The retrieval pipeline was designed in a modular manner so that embedding models and vector search strategies can be updated independently in future versions.

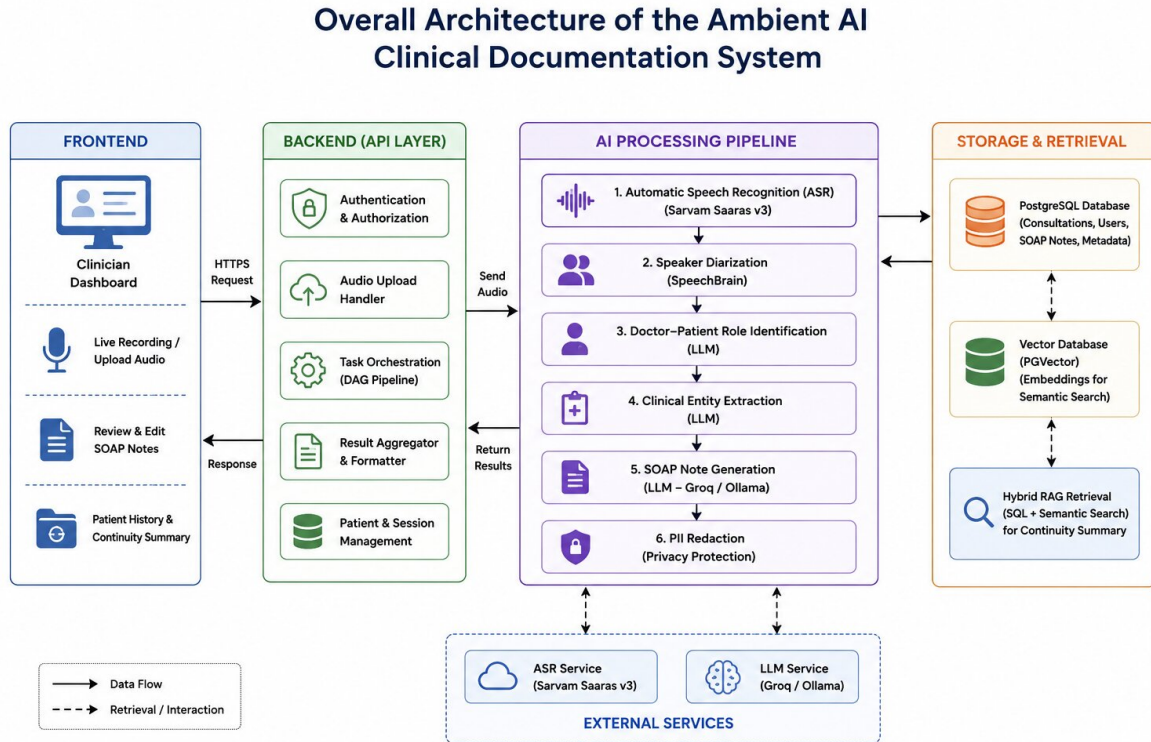


Figure 3.1: Overall Architecture of the Ambient AI Clinical Documentation System

The major modules included in the proposed framework are described below:

- **Authentication Module:** Handles clinician login, OTP verification, and secure access management using authentication and password protection mechanisms [9, 10].
- **Audio Ingestion Module:** Accepts live-recorded or uploaded consultation audio from clinicians through the frontend interface.
- **ASR and Diarization Module:** Converts consultation speech into textual transcripts and separates speaker segments for doctor and patient identification [1].
- **Role Detection Module:** Performs semantic analysis to identify doctor and patient speech roles within consultation transcripts.
- **Clinical Structuring Module:** Extracts important medical entities such as symptoms, medications, diagnoses, and treatment-related information from consultation text.

- **SOAP Generation Module:** Generates structured SOAP notes from the processed consultation transcript using Large Language Models [5, 2].
- **PII Redaction Module:** Removes sensitive patient-related information before database storage.
- **Hybrid RAG Module:** Retrieves semantically relevant historical consultation records using SQL retrieval and vector similarity search mechanisms [3, 7, 4].
- **Patient Management Module:** Maintains consultation history, continuity summaries, and patient-related records using PostgreSQL databases [12].

Figure 3.1 illustrates the interaction between frontend interfaces, backend APIs, AI processing modules, retrieval systems, and database components within the proposed Ambient AI platform.

3.2 Description of Target Users

The primary users of the proposed system are healthcare professionals such as doctors, clinicians, and medical practitioners who regularly perform patient consultations and maintain clinical records. The platform is particularly useful in hospitals and clinics where clinicians handle large numbers of consultations every day.

The system was designed with the objective of reducing manual documentation effort while improving the organization and accessibility of consultation records. Healthcare startups, telemedicine platforms, and clinical documentation services can also utilize the platform for generating structured medical summaries and continuity-aware patient records.

During frontend development, special attention was given to interface simplicity so that clinicians with limited technical expertise could easily upload consultation audio, review generated SOAP notes, and access historical patient summaries without complex workflows.

3.3 Advantages and Applications of Proposed System

The proposed system offers several advantages over conventional healthcare documentation workflows. By integrating Artificial Intelligence into consultation processing, the platform improves documentation efficiency, consultation consistency, and accessibility of patient records [3, 2].

The major advantages and applications of the proposed system are listed below:

- Reduces the amount of manual clinical documentation work performed by clinicians.

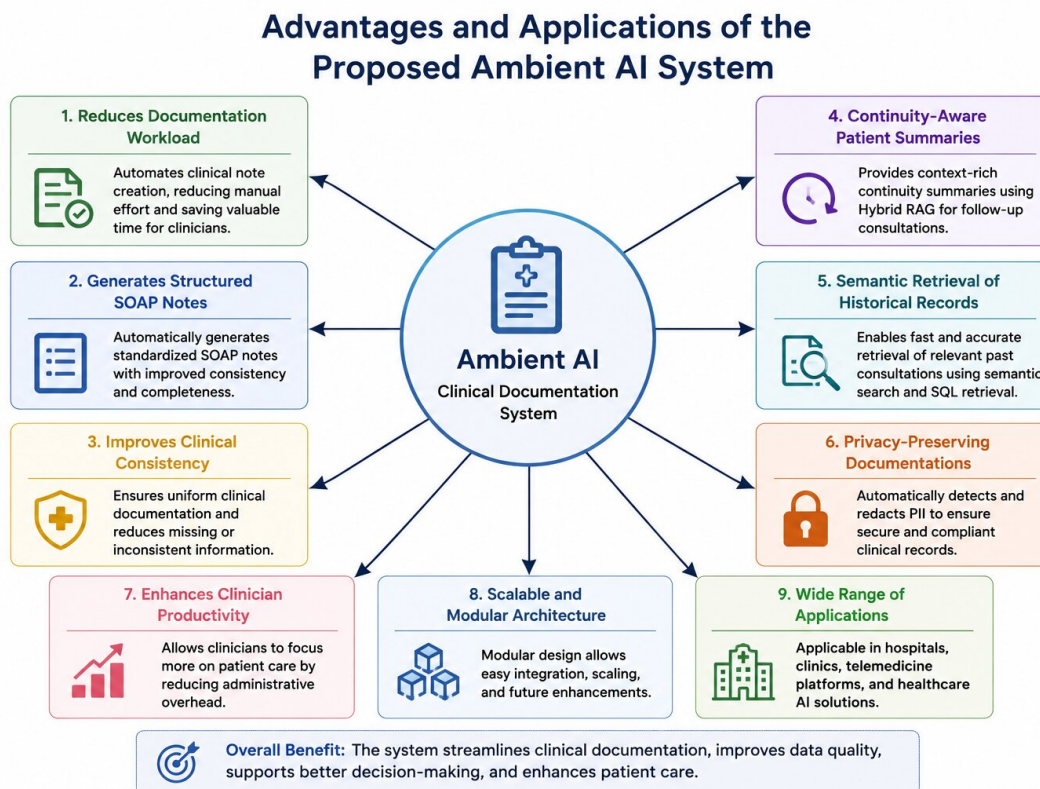


Figure 3.2: Advantages and Applications of the Proposed System

- Automatically generates structured SOAP notes from consultation conversations.
- Improves organization and consistency of consultation records.
- Supports continuity-aware retrieval using Hybrid RAG mechanisms.
- Reduces administrative workload and improves clinician productivity.
- Enables semantic retrieval of historical patient consultations.
- Performs automated PII redaction for privacy-preserving documentation.
- Can be integrated into hospitals, clinics, telemedicine systems, and healthcare AI platforms.

Figure 3.2 illustrates the major advantages and application areas of the proposed Ambient AI healthcare documentation system.

3.4 Scope (Boundary of Proposed System)

The scope of the proposed system includes consultation audio processing, speaker role identification, structured clinical entity extraction, SOAP note generation, patient history manage-

ment, and continuity-aware retrieval using Hybrid RAG mechanisms.

The platform is intended to function as a healthcare documentation assistance system and does not independently provide medical diagnosis or treatment recommendations. All generated outputs require clinician review before final usage or storage.

The current implementation mainly focuses on consultation processing and continuity retrieval workflows. Features such as integration with hospital Electronic Health Record (EHR) systems, large-scale multilingual deployment, and real-time streaming inference are considered future enhancements.

Figure 3.2 illustrates the major advantages and application areas of the proposed Ambient AI platform.

3.5 System Workflow

The proposed system includes workflow diagrams, architecture representations, and implementation illustrations to improve understanding of the processing pipeline and module interactions. These visual representations help explain the movement of data between frontend interfaces, backend services, AI models, and database systems.

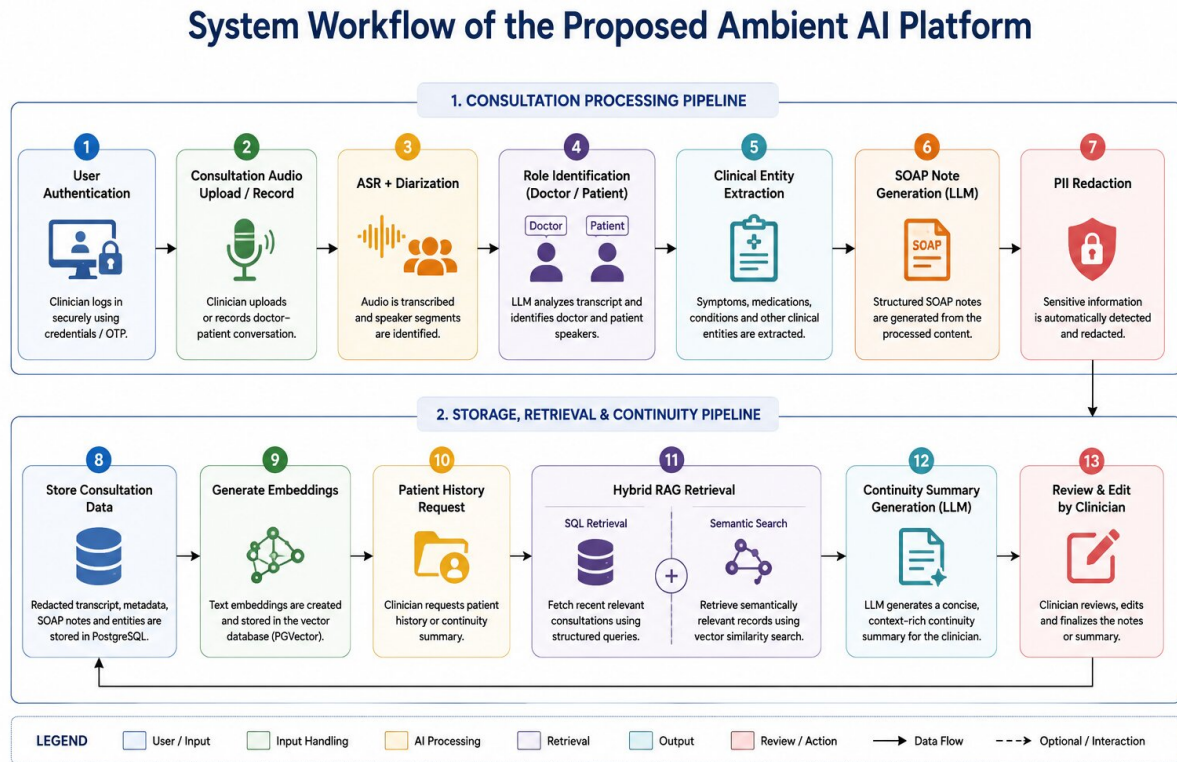


Figure 3.3: System Workflow of the Proposed Ambient AI Platform

The workflow begins with clinician authentication and consultation audio upload. The uploaded consultation is then forwarded through ASR, speaker diarization, semantic analysis, SOAP generation, and Hybrid RAG retrieval stages before being stored in the database. During implementation, modular execution of these stages simplified testing and allowed independent debugging of AI services.

The architecture also supports independent upgrading of retrieval systems, frontend interfaces, and AI models without affecting the overall workflow execution.

Figure 3.3 illustrates the operational workflow and overall data processing sequence used within the proposed Ambient AI clinical documentation platform.

Chapter 4

SYSTEM DESIGN

This chapter explains the system design and implementation workflow of the proposed Ambient AI clinical documentation platform. It describes the architectural design, interaction between modules, database organization, workflow execution, and processing pipeline used during implementation. The chapter also discusses the integration of Automatic Speech Recognition (ASR), Large Language Models (LLMs), Hybrid Retrieval-Augmented Generation (RAG), and database management components within the developed system architecture [1, 2, 3].

4.1 Architecture of the system

The proposed Ambient AI platform follows a modular client–server architecture consisting of frontend interfaces, backend APIs, AI processing services, retrieval modules, and database systems. The architecture was designed in a modular manner so that individual components could be independently developed, tested, and upgraded without affecting the complete workflow.

The consultation workflow begins when a clinician authenticates into the platform and uploads or records consultation audio using the frontend interface. The uploaded consultation audio is forwarded to backend processing services where ASR and speaker diarization models generate transcripts and identify speaker segments [1]. During implementation, handling different audio qualities and speaker overlaps required additional preprocessing to improve transcription consistency.

The generated transcript is then processed through semantic analysis and LLM-based modules to identify doctor and patient speech roles, extract important clinical entities, and generate structured SOAP notes [2, 6, 5]. Privacy-preserving PII redaction is performed before storing records in the database.

To improve continuity during follow-up consultations, the processed consultation summaries are additionally passed through the Hybrid RAG retrieval pipeline. This module combines SQL-based recency retrieval with semantic vector similarity search to retrieve contextually relevant historical consultation information [3, 7, 4]. The processed outputs are finally stored within the relational database for future retrieval and patient consultation management [12].

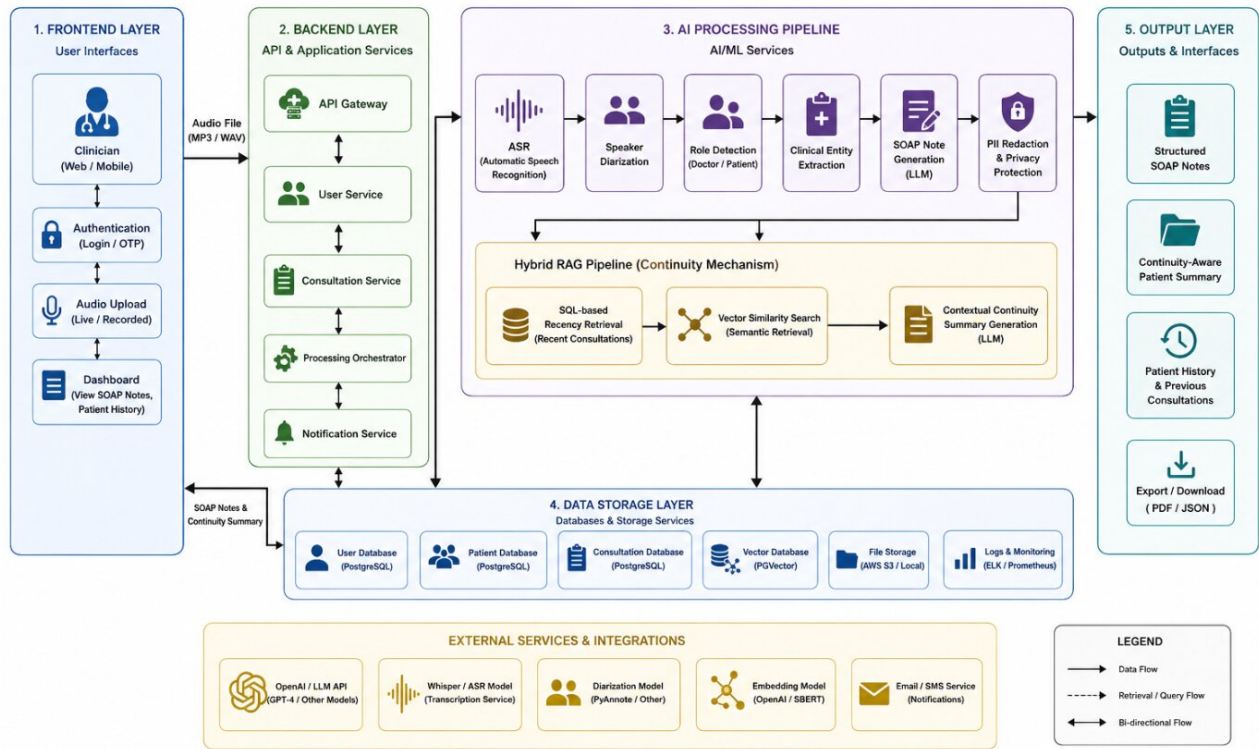


Figure 4.1: System Architecture of the Ambient AI Platform

Figure 4.1 illustrates the interaction between frontend interfaces, backend APIs, AI processing modules, retrieval services, and database components used within the proposed system.

Algorithm 1 Clinical Consultation Processing Pipeline

Require: Consultation audio input

Ensure: Structured SOAP notes and continuity summary

- 1: Authenticate clinician credentials
 - 2: Upload consultation audio
 - 3: Perform ASR transcription
 - 4: Perform speaker diarization
 - 5: Identify doctor and patient speakers
 - 6: Extract clinical entities from transcript
 - 7: Generate SOAP notes using LLM
 - 8: Perform PII redaction
 - 9: Generate semantic embeddings
 - 10: Retrieve historical consultations using Hybrid RAG
 - 11: Store processed consultation data
 - 12: Return SOAP notes and continuity summary
-

Algorithm 1 describes the complete consultation processing workflow used for generating structured SOAP summaries and continuity-aware consultation retrieval.

4.2 Class Diagram (with brief explanation)

The class diagram represents the structural organization of the major software entities and modules used within the proposed Ambient AI system. The implementation includes classes related to clinician authentication, consultation processing, transcript management, SOAP generation, retrieval pipelines, and database persistence.

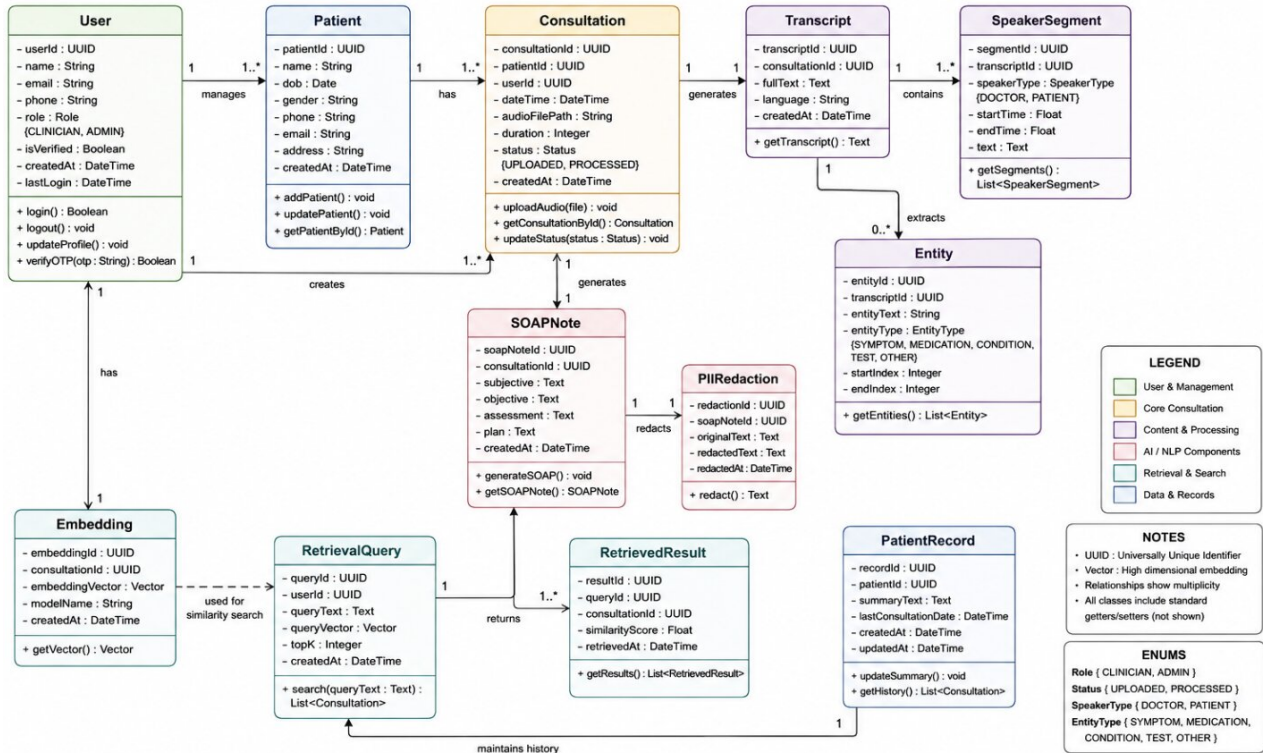


Figure 4.2: Class Diagram of Ambient AI System

The major classes used within the system are described below:

- **User Class:** Maintains clinician authentication details, profile information, and login credentials.
- **Consultation Class:** Stores consultation metadata, timestamps, and uploaded audio references.
- **Transcript Class:** Maintains ASR-generated consultation transcripts and diarized speaker segments.

- **SOAPNote Class:** Stores structured SOAP summaries generated from consultation transcripts.
- **Embedding Class:** Maintains semantic vector embeddings used for retrieval operations.
- **PatientRecord Class:** Stores consultation history and continuity-related patient information.

Figure 4.2 illustrates the relationship between major system entities and processing modules.

4.3 Sequence diagram

The sequence diagram represents the chronological interaction between clinicians, frontend interfaces, backend services, AI modules, and database systems during consultation processing.

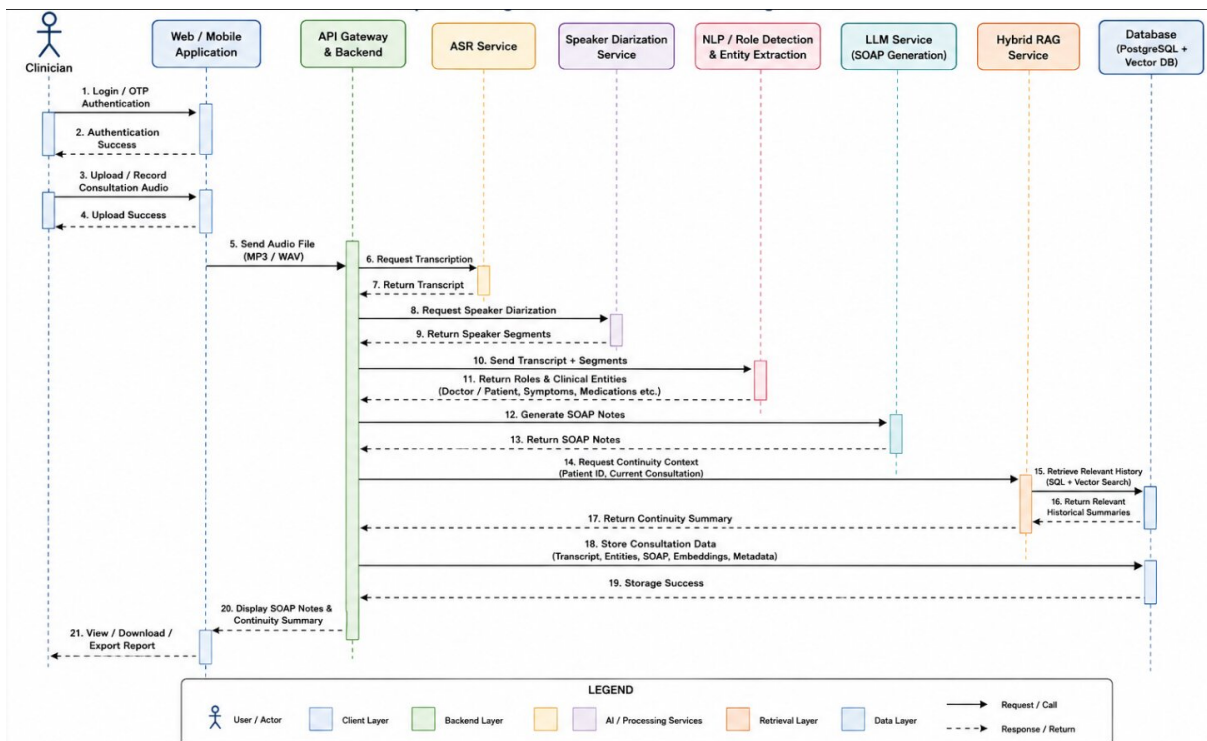


Figure 4.3: Sequence Diagram for Consultation Processing

The sequence begins when the clinician uploads consultation audio using the frontend dashboard. The backend forwards the uploaded audio to ASR and speaker diarization services for transcript generation and speaker separation [1]. The generated transcript is then processed by semantic analysis and SOAP generation modules to produce structured clinical summaries [2].

After SOAP generation and PII redaction, the processed consultation information is stored within the database and later utilized by the Hybrid RAG retrieval pipeline during future consultations [3]. During implementation, separating the workflow into modular API calls simplified debugging and backend integration.

Figure 4.3 illustrates the communication flow and execution sequence between major system components.

4.4 ER diagram and schema

The Entity Relationship (ER) diagram represents the database schema and relationships maintained within the proposed Ambient AI platform. The database structure was designed to support consultation storage, transcript persistence, SOAP summaries, semantic embedding retrieval, and patient history management.

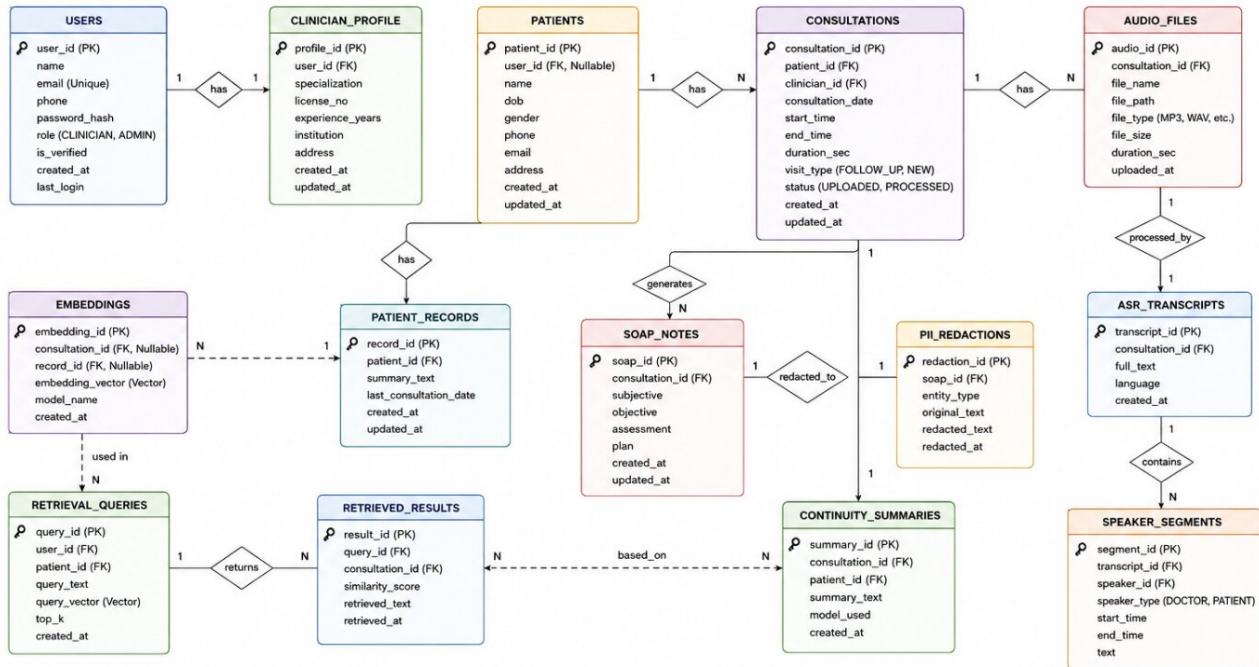


Figure 4.4: Entity Relationship Diagram of Ambient AI Database

The primary entities used within the database include Users, Patients, Consultations, Transcripts, SOAP Notes, Embeddings, and Continuity Summaries. Foreign key relationships are used to maintain consistency between patient records and consultation histories [12].

The database schema was designed to support efficient retrieval of consultation history while also maintaining compatibility with vector-based semantic search operations.

Figure 4.4 illustrates the database entities and relationships used within the proposed system.

4.5 State transition diagram

The state transition diagram represents the movement of consultation data between different processing stages during execution of the Ambient AI workflow. The consultation progresses through multiple states including audio upload, transcription, speaker diarization, entity extraction, SOAP generation, retrieval processing, and database persistence.

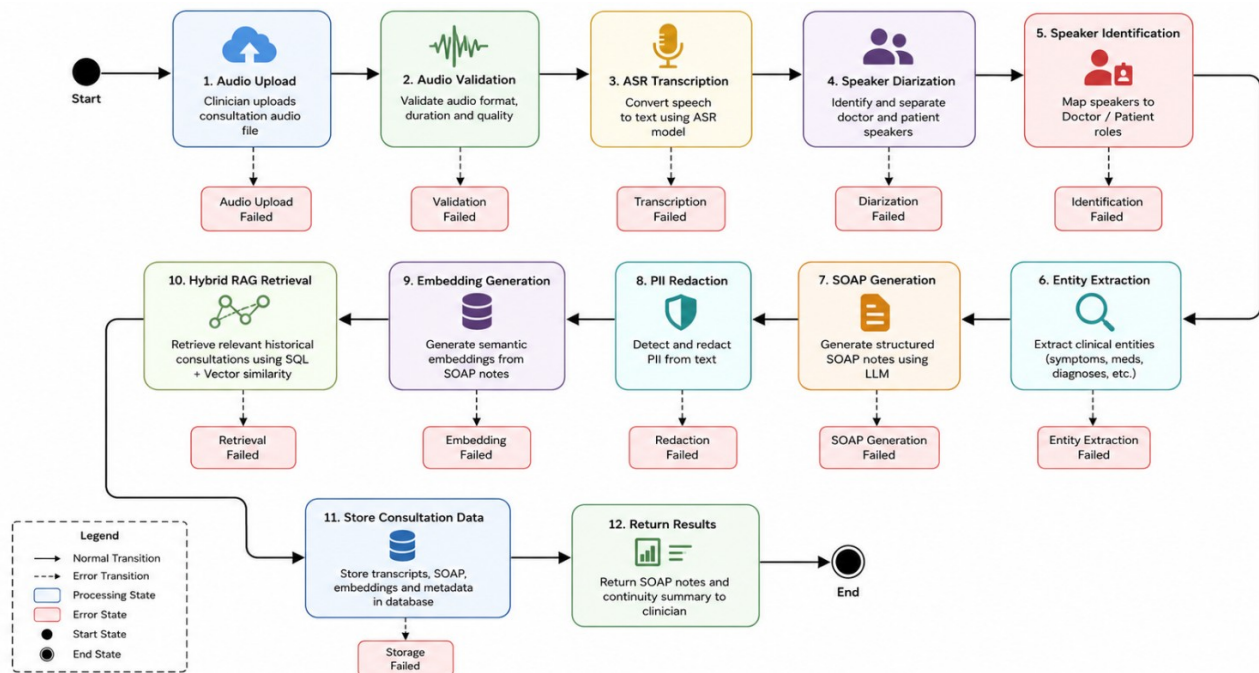


Figure 4.5: State Transition Diagram of Consultation Processing

The transition between states ensures proper synchronization between AI modules, retrieval pipelines, and storage services during consultation processing. During implementation, separating the workflow into independent processing stages improved modularity and simplified fault handling within the backend services.

Figure 4.5 illustrates the consultation state transitions involved during clinical documentation generation.

4.6 Data Set Description

The proposed system primarily utilizes doctor–patient consultation audio recordings as the input dataset. The consultation audio contains multilingual healthcare conversations involving symptoms, diagnoses, medications, treatment discussions, and follow-up recommendations.

The dataset used during testing and validation included consultation samples with different durations, speaker variations, and clinical contexts. Audio preprocessing operations such as segmentation and noise reduction were applied before ASR processing to improve transcription quality [1].

The generated transcripts were further utilized for semantic analysis, SOAP note generation, clinical entity extraction, and continuity-aware retrieval operations. Embedding vectors generated from historical consultation summaries were stored within the vector database to support Hybrid RAG-based retrieval during follow-up consultations [3, 7, 4].

The dataset was additionally used for evaluating transcription quality, speaker identification accuracy, SOAP consistency, retrieval relevance, and continuity summarization performance across different consultation scenarios.

Chapter 5

IMPLEMENTATION

This chapter describes the implementation details of the proposed Ambient AI clinical documentation platform. The implementation focuses on integrating Artificial Intelligence driven healthcare documentation workflows using Automatic Speech Recognition (ASR), speaker diarization, semantic processing, Large Language Models (LLMs), and Hybrid Retrieval-Augmented Generation (RAG) [1, 2, 3]. The chapter explains the implementation methodology, major software modules, processing workflows, and algorithmic pipeline used for generating structured clinical summaries from doctor–patient conversations.

5.1 Proposed Methodology

The proposed methodology follows a multi-stage AI processing pipeline designed to convert raw consultation audio into structured clinical documentation. The implementation combines frontend interfaces, backend APIs, AI inference modules, vector retrieval systems, and relational databases to achieve automated clinical documentation generation [11, 12].

The workflow begins with clinician authentication and consultation audio upload through the frontend interface. The uploaded audio is forwarded to the backend processing pipeline, where ASR models generate textual transcripts from consultation speech [1]. Speaker diarization models separate doctor and patient speech segments for role identification and structured processing.

The generated transcript is then processed using semantic analysis and Large Language Models to extract symptoms, diagnoses, medications, and treatment-related entities. Structured SOAP notes are generated from the extracted information, followed by privacy-preserving PII redaction before storage [2, 6, 5].

The system further implements a Hybrid RAG mechanism that combines SQL-based recency retrieval with semantic vector similarity search to retrieve continuity-aware patient summaries during follow-up consultations [3, 7, 4].

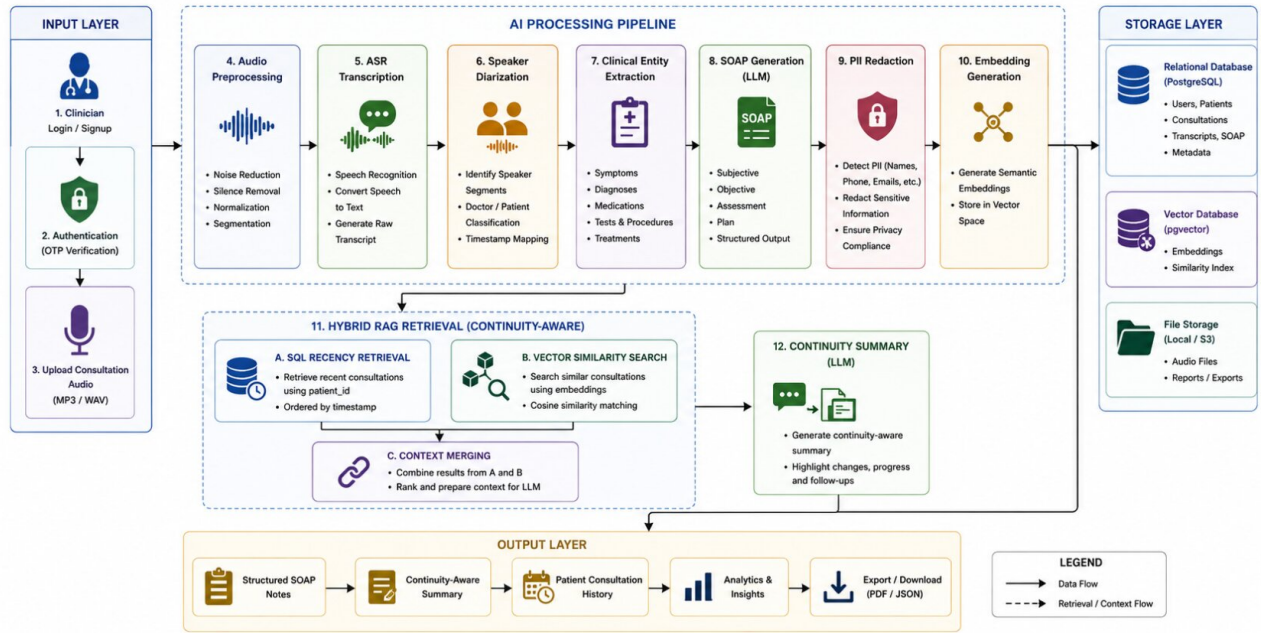


Figure 5.1: Implementation Workflow of Ambient AI System

Figure 5.1 illustrates the complete implementation workflow followed by the proposed Ambient AI platform.

5.2 Description of Modules

The proposed Ambient AI platform is divided into multiple implementation modules, where each module performs a dedicated operation within the clinical documentation workflow.

Authentication Module

Module Name : authentication()

Input : Clinician credentials and OTP verification request.

Output : Authenticated clinician session.

This module handles secure clinician authentication using email verification and OTP-based login validation [9, 10]. The module ensures secure access management before consultation processing begins.

Audio Processing Module

Module Name : audio_processing()

Input : Consultation audio recording.

Output : Processed audio transcript and speaker segments.

This module performs Automatic Speech Recognition and speaker diarization on uploaded consultation audio [1]. Noise reduction and preprocessing techniques are applied before transcription generation.

Clinical Entity Extraction Module

Module Name : entity_extraction()

Input : Consultation transcript.

Output : Extracted clinical entities.

This module extracts symptoms, medications, diagnoses, laboratory tests, and treatment-related entities using semantic analysis and LLM-based processing [6, 2].

SOAP Generation Module

Module Name : soap_generation()

Input : Structured consultation transcript.

Output : Structured SOAP notes.

This module generates Subjective, Objective, Assessment, and Plan (SOAP) summaries from consultation transcripts using Large Language Models [5, 2].

PII Redaction Module

Module Name : pii_redaction()

Input : Generated consultation summaries.

Output : Privacy-compliant consultation records.

This module removes Personally Identifiable Information (PII) such as patient names, phone numbers, and sensitive identifiers before database storage.

Hybrid RAG Retrieval Module

Module Name : hybrid_rag()

Input : Current consultation summary and patient identifier.

Output : Continuity-aware historical consultation summary.

This module performs retrieval operations using SQL recency search and semantic vector similarity matching for retrieving relevant historical consultation records [3, 7, 4].

Algorithm 2 Clinical Consultation Processing Algorithm

Require: Consultation audio recording

Ensure: Structured SOAP summary and continuity retrieval

- 1: Authenticate clinician credentials
 - 2: Upload consultation audio
 - 3: Perform audio preprocessing
 - 4: Generate transcript using ASR
 - 5: Perform speaker diarization
 - 6: Identify doctor and patient speakers
 - 7: Extract clinical entities
 - 8: Generate SOAP notes using LLM
 - 9: Perform PII redaction
 - 10: Generate semantic embeddings
 - 11: Retrieve historical consultations using Hybrid RAG
 - 12: Store consultation records in database
 - 13: Return SOAP notes and continuity summary
-

Algorithm 2 describes the implementation workflow used for consultation processing, SOAP generation, and continuity-aware retrieval within the Ambient AI system.

The module interaction workflow begins with clinician authentication and consultation audio upload through the frontend interface. The uploaded audio is forwarded to the ASR and speaker diarization modules for transcription and speaker separation [1]. The generated transcript is then processed by the clinical entity extraction and SOAP generation modules to produce structured consultation summaries [2, 5]. Privacy-preserving PII redaction is performed before storing the processed records within the relational database.

The generated consultation summaries are additionally processed through the Hybrid RAG retrieval pipeline, where SQL-based recency retrieval and semantic vector similarity search are combined to retrieve relevant historical consultation records [3, 7, 4]. The retrieved contextual information is then used to generate continuity-aware summaries for follow-up consultations. The modular architecture allows independent execution and scalability of individual AI services without affecting the overall system workflow.

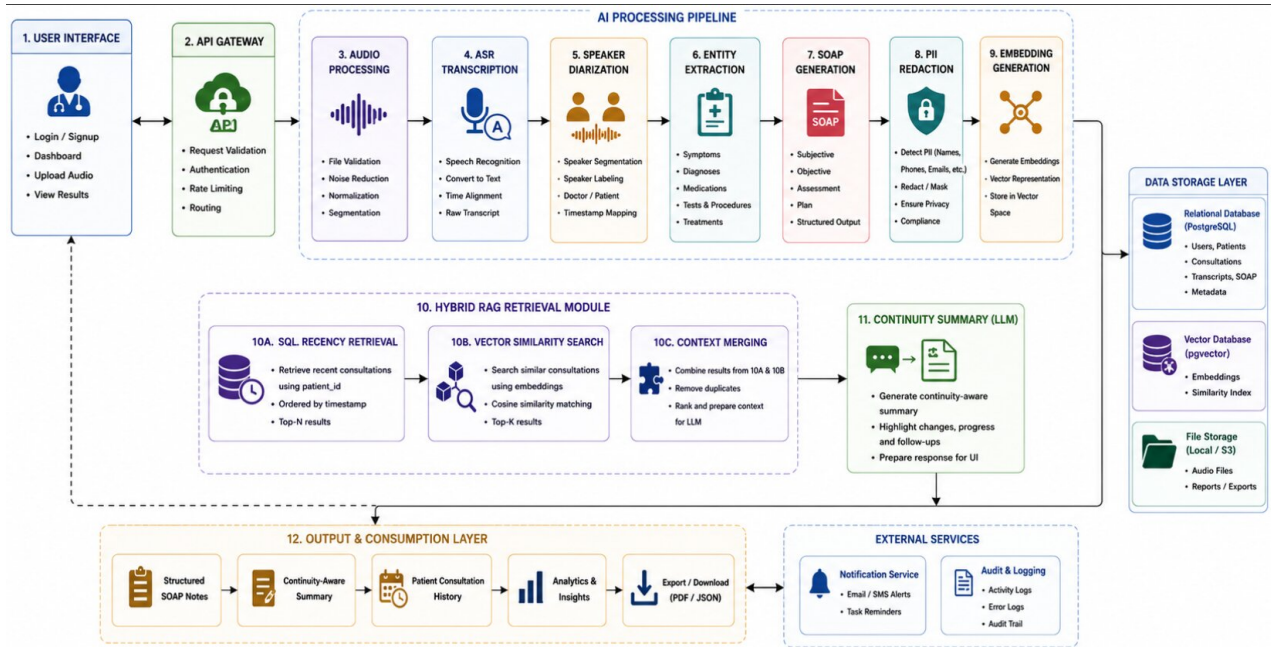


Figure 5.2: Module Interaction Workflow

Figure 5.2 illustrates the interaction between major implementation modules used within the proposed system. ““

Chapter 6

TESTING

This chapter describes the testing procedures carried out to validate the functionality, reliability, and performance of the proposed Ambient AI clinical documentation platform. Different testing methods were used to evaluate consultation processing, SOAP note generation, Hybrid RAG retrieval, database operations, and frontend–backend communication. Testing was performed on both individual modules and the complete system workflow under different operational conditions [8].

6.1 Test Plan and Test Cases

The proposed system was evaluated using multiple testing approaches including unit testing, integration testing, functional testing, and acceptance testing. These testing procedures were performed to verify the correctness of AI processing pipelines, retrieval mechanisms, frontend operations, and API communication workflows.

During implementation, testing was carried out continuously after integration of every major module to identify errors at early stages and improve overall system stability.

The major objectives of the testing process are listed below:

- Verify successful clinician authentication and session handling.
- Validate consultation audio upload and preprocessing functionality.
- Evaluate ASR transcription and speaker diarization performance [1].
- Verify extraction of clinical entities and SOAP note generation [5, 2].
- Validate Hybrid RAG retrieval and continuity summarization [3, 7, 4].
- Ensure proper storage and retrieval of consultation records.
- Verify frontend responsiveness and backend API communication.
- Ensure privacy compliance through PII redaction testing.

6.2 Acceptance Test Plan and Test Cases

Acceptance testing was performed to verify whether the developed system satisfied the expected functional requirements and user workflows. The testing process mainly focused on evaluating the complete consultation pipeline from clinician authentication to SOAP note generation and continuity retrieval.

Table 6.1: Acceptance Test Cases

ID	Test Scenario	Expected Output	Actual Result	Status
AT1	Clinician login with valid credentials	Successful authentication	Authentication successful	Pass
AT2	Upload consultation audio	Audio uploaded successfully	Upload successful	Pass
AT3	Generate transcript from consultation audio	Accurate transcript generation	Transcript generated	Pass
AT4	Perform speaker diarization	Correct speaker separation	Speakers identified correctly	Pass
AT5	Generate SOAP notes	Structured SOAP summary generated	SOAP notes generated	Pass
AT6	Retrieve historical consultation records	Relevant continuity summary generated	Retrieval successful	Pass
AT7	Perform PII redaction	Sensitive information removed	PII removed successfully	Pass

The acceptance testing results confirmed that the developed system successfully satisfied the major functional requirements related to consultation processing, SOAP generation, retrieval operations, and privacy-preserving data storage.

6.3 Unit Test Plan and Test Cases

Unit testing was performed on individual software modules to verify the correctness of independent system components. Each module was tested separately using predefined inputs and expected outputs before integration into the complete workflow.

Table 6.2: Unit Test Cases

ID	Module Tested	Expected Result	Actual Result	Status
UT1	Authentication Module	User login validation	Authentication validated	Pass
UT2	Audio Processing Module	Successful audio preprocessing	Audio processed correctly	Pass
UT3	ASR Module	Accurate speech transcription	Transcript generated	Pass
UT4	Speaker Diarization Module	Correct speaker segmentation	Speaker separation successful	Pass
UT5	Entity Extraction Module	Extraction of clinical entities	Clinical entities extracted	Pass
UT6	SOAP Generation Module	Structured SOAP notes generated	SOAP notes generated	Pass
UT7	PII Redaction Module	Sensitive information masked	PII removed successfully	Pass
UT8	Hybrid RAG Retrieval Module	Relevant consultation retrieval	Retrieval successful	Pass
UT9	Database Storage Module	Consultation record persistence	Data stored successfully	Pass

The unit testing results showed that the individual modules performed correctly under expected input conditions and maintained reliable interaction with connected services.

6.4 Integration Testing

Integration testing was carried out to validate communication between frontend interfaces, backend APIs, AI processing modules, retrieval services, and database systems [11, 12]. The purpose of this testing phase was to ensure smooth data flow between all connected modules.

During implementation, integration testing helped identify synchronization issues between transcription services, SOAP generation modules, and database persistence operations.

Table 6.3: Integration Testing Results

ID	Integrated Modules	Testing Objective	Status
IT1	Frontend and Authentication API	Validate clinician login workflow	Pass
IT2	Audio Upload and ASR Pipeline	Validate audio processing workflow	Pass
IT3	ASR and SOAP Generation Modules	Validate transcript-to-SOAP workflow	Pass
IT4	Retrieval Module and Database	Validate Hybrid RAG retrieval workflow	Pass
IT5	Backend and Frontend Dashboard	Validate dashboard visualization workflow	Pass

The integration testing results confirmed successful interaction and synchronization between the major modules implemented within the proposed Ambient AI system.

6.5 Performance Testing

Performance testing was conducted to evaluate system responsiveness, consultation processing speed, and retrieval efficiency under different execution conditions.

The testing process focused on measuring the responsiveness of transcription generation, SOAP note creation, retrieval operations, and database persistence during consultation processing workflows.

Table 6.4: Performance Testing Results

Operation	Observed Performance	Result
Audio Upload	Successful upload within acceptable delay	Pass
ASR Processing	Fast transcription generation	Pass
SOAP Generation	Structured summary generated efficiently	Pass
Hybrid RAG Retrieval	Relevant consultation retrieval within short duration	Pass
Database Storage	Consultation records stored successfully	Pass

The testing results demonstrated that the proposed system performed efficiently during consultation processing, semantic retrieval, and structured clinical documentation generation workflows.

““latex

Chapter 7

RESULTS & DISCUSSIONS

This chapter presents the results obtained from the implementation and testing of the proposed Ambient AI clinical documentation platform. The generated outputs were evaluated based on speaker identification, SOAP note generation, Hybrid RAG retrieval accuracy, front-end usability, and overall system performance. The chapter also discusses the interpretation of obtained results and analyzes the effectiveness of the proposed system in automating health-care documentation workflows [1, 2, 3].

7.1 Doctor Voice Enrollment Results

The proposed system successfully implemented a doctor voice enrollment mechanism during clinician registration. During the enrollment process, clinicians first select the languages in which they are fluent and later provide sample voice recordings that are processed and stored for personalized speaker identification during consultation analysis.

The multilingual language selection mechanism allows the system to support doctors communicating in multiple regional languages during consultation sessions. This improves speaker adaptation and enables better multilingual consultation handling during ASR and diarization processing.

After language selection, clinicians complete voice enrollment by recording guided voice samples. The enrolled voice samples are utilized during speaker diarization to improve separation between doctor and patient speech segments. This personalized voice enrollment mechanism improved speaker recognition consistency and reduced ambiguity during consultation processing.

The enrollment interface additionally provides secure clinician registration, audio recording functionality, waveform visualization, voice verification, and speaker profile creation for maintaining voice identity mappings within the system.

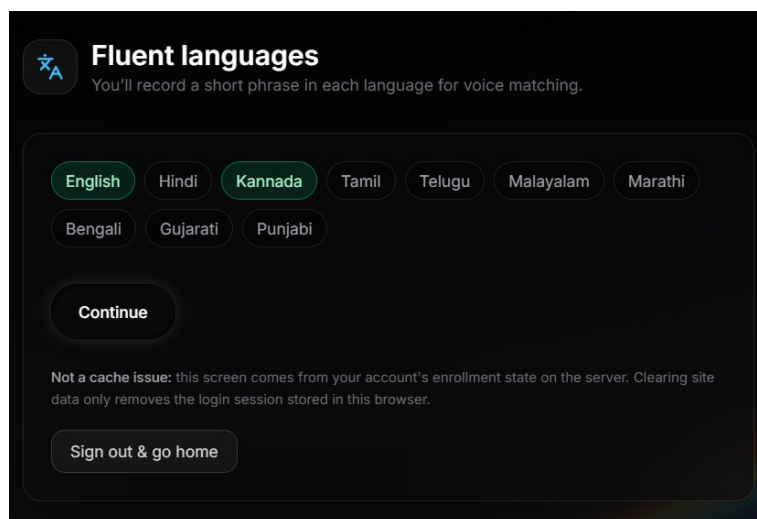


Figure 7.1: Doctor Language Selection During Voice Enrollment

Figure 7.1 illustrates the multilingual language selection interface used during doctor voice enrollment.

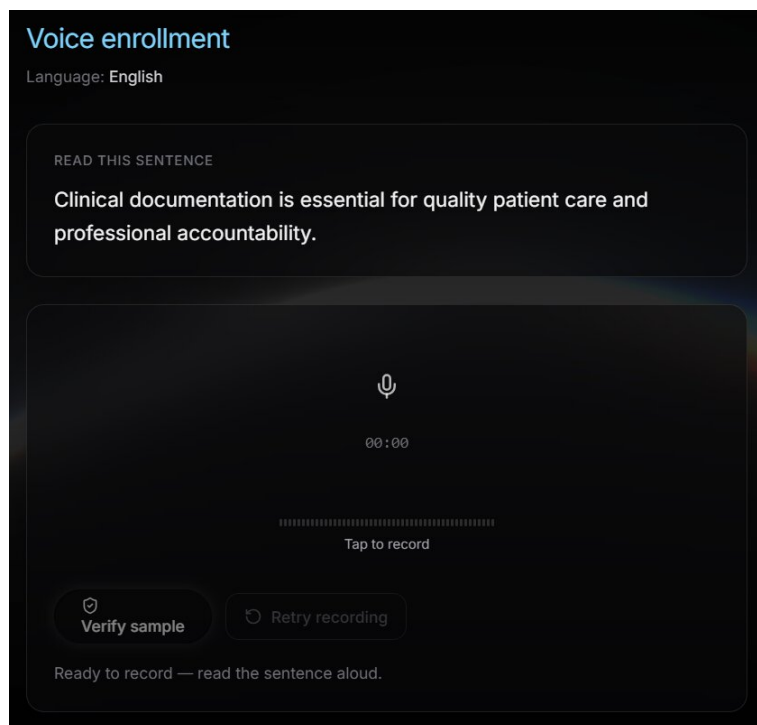


Figure 7.2: Doctor Voice Recording and Verification Interface

Figure 7.2 illustrates the doctor voice recording and verification interface used for registering clinician voice samples during system onboarding.

7.2 Dashboard and Consultation Interface Results

The frontend dashboard successfully provided clinicians with interfaces for consultation upload, live consultation management, SOAP note review, continuity summary visualization, and patient history management. The responsive interface improved usability and simplified interaction with the AI processing workflow.

The dashboard additionally provides consultation management tools including audio upload, generated transcript review, SOAP editing, continuity-aware retrieval visualization, and patient history access through a unified interface.

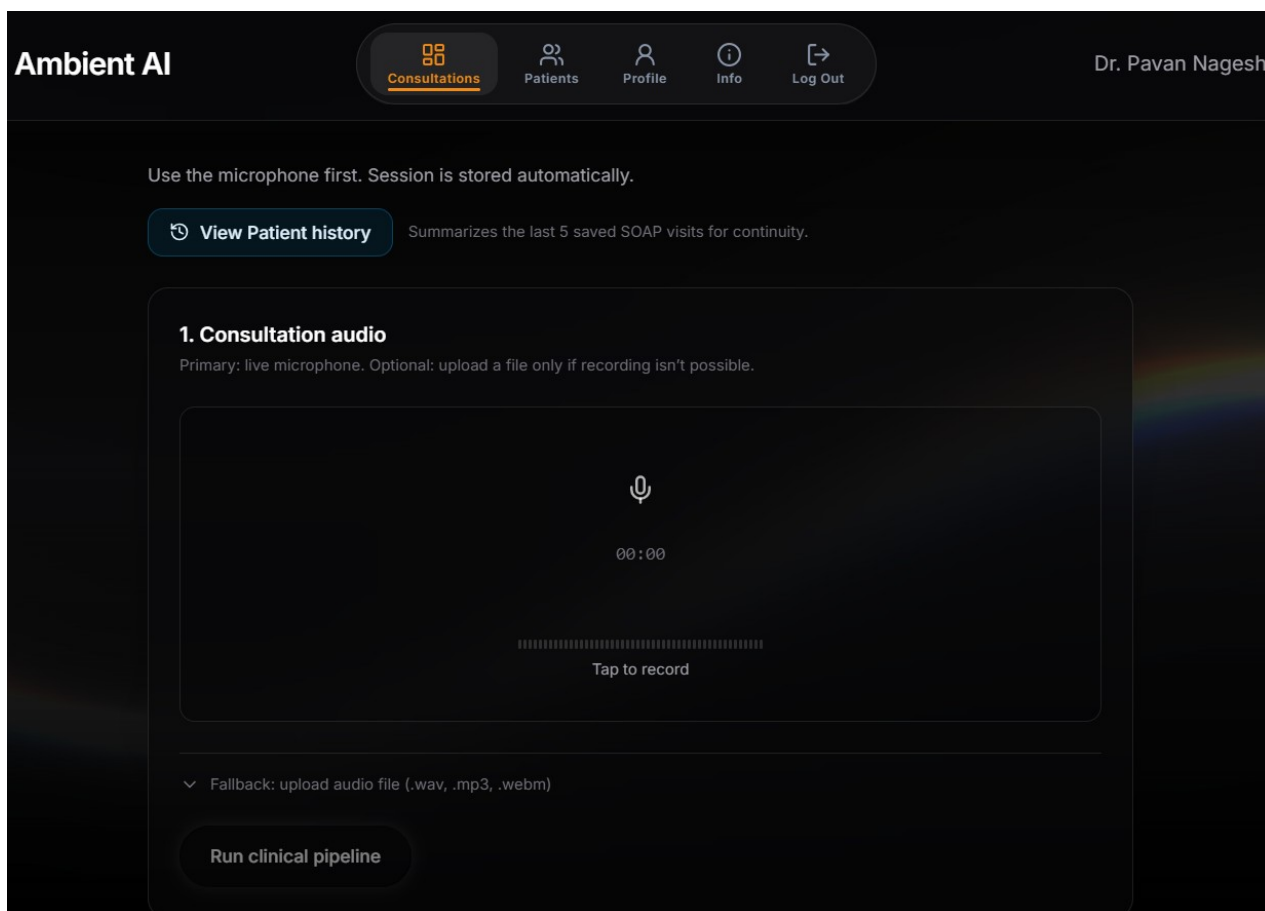


Figure 7.3: Frontend Dashboard and Consultation Interface

Figure 7.3 illustrates the clinician dashboard interface used for consultation management, SOAP review, and patient history visualization.

7.3 Speaker Diarization Results

The speaker diarization module successfully identified and separated doctor and patient speech segments from consultation recordings. Timestamp mapping and semantic speaker identification improved the structuring of consultation transcripts.

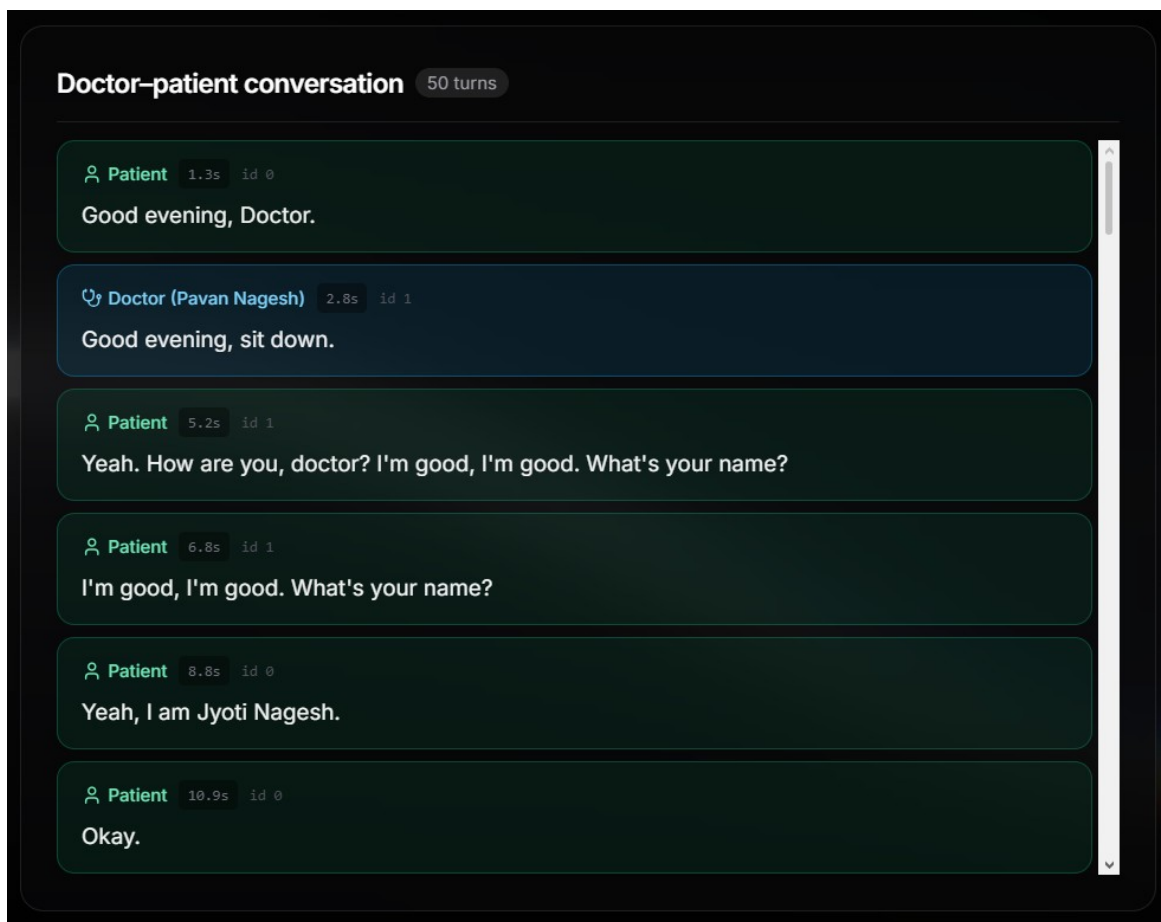


Figure 7.4: Speaker Diarization Result

Figure 7.4 illustrates the separation of doctor and patient speaker segments generated during consultation processing.

7.4 SOAP Note Generation Results

The Large Language Model based SOAP generation module successfully converted unstructured consultation transcripts into structured clinical documentation consisting of Subjective, Objective, Assessment, and Plan sections [2, 5].

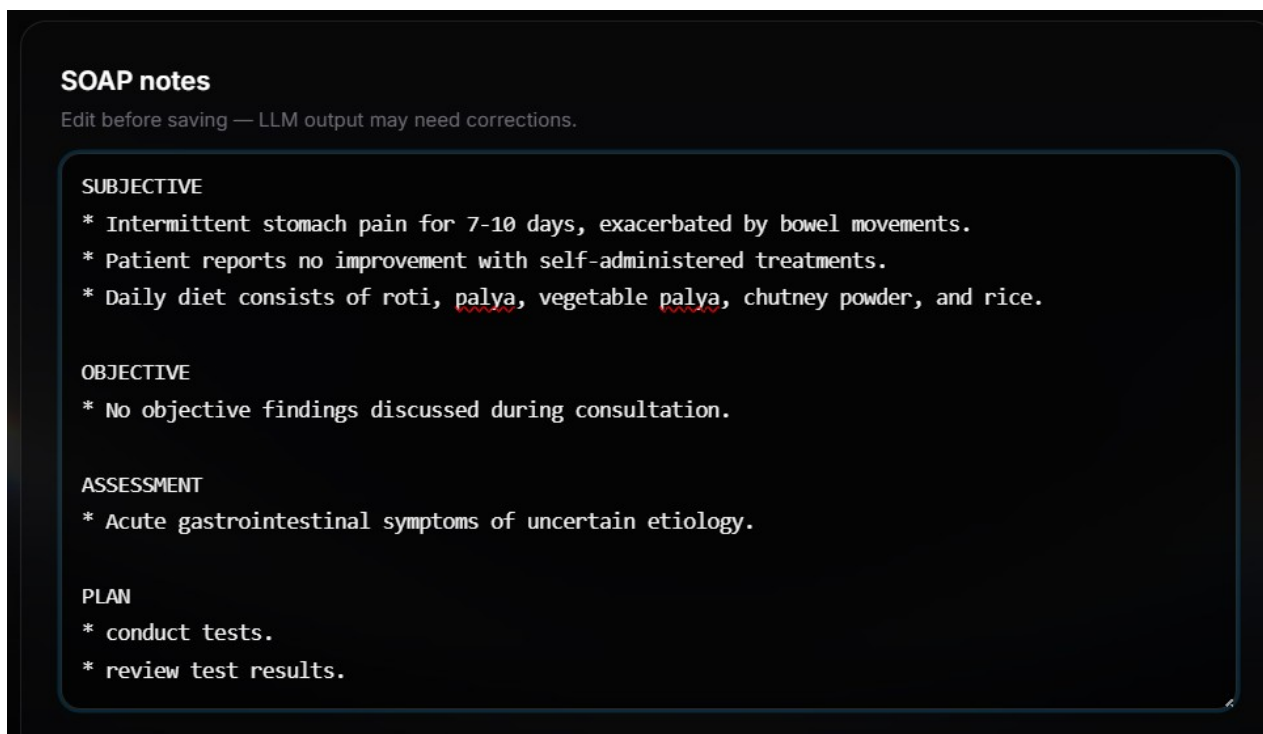
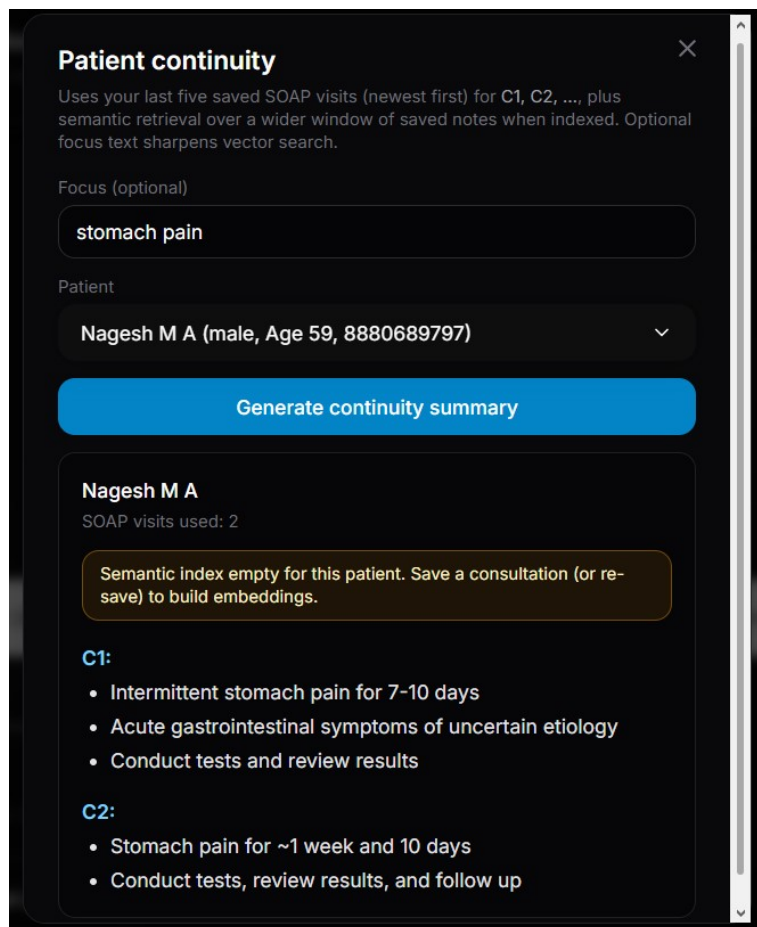


Figure 7.5: Generated SOAP Notes

Figure 7.5 illustrates the structured SOAP notes generated from consultation transcripts. The generated summaries provide organized clinical documentation for efficient healthcare record management.

7.5 Hybrid RAG Retrieval Results

The Hybrid Retrieval-Augmented Generation (RAG) module successfully retrieved relevant historical consultation summaries using SQL recency retrieval and semantic vector similarity search [3, 7, 4]. The retrieved contextual information improved continuity-aware summarization during follow-up consultations.



Patient continuity ✕

Uses your last five saved SOAP visits (newest first) for C1, C2, ..., plus semantic retrieval over a wider window of saved notes when indexed. Optional focus text sharpens vector search.

Focus (optional)

stomach pain

Patient

Nagesh M A (male, Age 59, 8880689797) ▾

Generate continuity summary

Nagesh M A
SOAP visits used: 2

Semantic index empty for this patient. Save a consultation (or re-save) to build embeddings.

C1:

- Intermittent stomach pain for 7-10 days
- Acute gastrointestinal symptoms of uncertain etiology
- Conduct tests and review results

C2:

- Stomach pain for ~1 week and 10 days
- Conduct tests, review results, and follow up

Figure 7.6: Hybrid RAG Retrieval Result

Figure 7.6 illustrates the retrieval of contextually relevant historical consultation records using the Hybrid RAG pipeline.

7.6 Performance Analysis

The proposed Ambient AI system demonstrated efficient performance during consultation processing and continuity retrieval operations. The integration of modular AI pipelines enabled scalable execution of speaker diarization, SOAP generation, and semantic retrieval workflows.

Table 7.1: Performance Analysis Results

Operation	Observed Result	Status
Voice Enrollment	Accurate clinician voice registration	Successful
Language Selection	Successful multi-lingual clinician onboarding	Successful
Speaker Diarization	Correct speaker identification	Successful
SOAP Generation	Structured clinical summary generation	Successful
Hybrid RAG Retrieval	Relevant consultation retrieval	Successful
PII Redaction	Sensitive information removal	Successful
Database Storage	Reliable consultation persistence	Successful
Frontend Responsiveness	Smooth clinician interaction	Successful

The performance analysis indicates that the proposed system effectively automates healthcare documentation workflows while maintaining reliable retrieval, structured summarization, speaker identification, and continuity-aware consultation management.

7.7 Discussion

The obtained results demonstrate that the proposed Ambient AI platform successfully integrates speaker diarization, semantic analysis, Large Language Models, and Hybrid RAG retrieval into a unified healthcare documentation workflow [1, 2, 3]. The generated SOAP notes improved organization of clinical records and reduced manual documentation effort for clinicians.

The implemented doctor voice enrollment mechanism improved speaker identification accuracy by enabling personalized clinician voice recognition during consultation analysis. The multilingual language selection and guided enrollment process additionally improved support for regional language consultations and multilingual healthcare workflows.

The Hybrid RAG mechanism significantly improved continuity-aware summarization by retrieving contextually relevant historical consultations during follow-up visits [3, 7]. The modular architecture additionally enabled scalable integration of AI processing services and retrieval pipelines.

The proposed system can be effectively utilized within hospitals, telemedicine platforms, healthcare startups, and clinical documentation systems for improving consultation management and reducing administrative workload in healthcare environments. ““

Chapter 8

CONCLUSIONS AND FUTURE SCOPE

This chapter presents the conclusions derived from the implementation and evaluation of the proposed Ambient AI clinical documentation platform. It also discusses the future enhancements and possible extensions that can further improve the capabilities and scalability of the system.

8.1 Conclusion

The proposed Ambient AI system successfully demonstrated the automation of clinical documentation workflows using Artificial Intelligence driven healthcare technologies. The system integrated Automatic Speech Recognition (ASR), speaker diarization, semantic analysis, Large Language Models (LLMs), and Hybrid Retrieval-Augmented Generation (RAG) into a unified platform for generating structured clinical documentation from doctor–patient consultations [1, 2, 3].

The implemented system successfully processed consultation audio recordings, generated accurate transcripts, identified speaker segments, extracted clinical entities, and produced structured SOAP notes [5, 6]. The Hybrid RAG retrieval mechanism additionally enabled continuity-aware consultation summarization by retrieving contextually relevant historical patient records using SQL retrieval and semantic vector similarity search [3, 7, 4].

The proposed platform also incorporated privacy-preserving PII redaction mechanisms to ensure secure handling of sensitive patient information before storage. The frontend dashboard provided clinicians with an interactive interface for consultation management, SOAP review, and patient history retrieval.

The obtained testing and performance results demonstrated that the proposed system effectively reduces manual documentation effort, improves consultation organization, enhances continuity-aware retrieval, and supports scalable integration of AI-driven healthcare documentation services.

8.2 Future Scope

Although the proposed Ambient AI system achieved successful automation of healthcare documentation workflows, several enhancements can be incorporated in future versions to improve functionality, scalability, and deployment capabilities.

Future work may include real-time streaming transcription and live SOAP generation during ongoing consultations to support real-time clinical assistance. The system can also be extended with multilingual ASR and multilingual LLM support for handling consultations conducted in regional languages [1, 2].

Integration with Electronic Health Record (EHR) systems and hospital management platforms can further improve interoperability and clinical adoption. Additional AI models can also be integrated for medical coding, prescription generation, diagnostic assistance, and healthcare analytics.

Future implementations may additionally focus on improving retrieval optimization, reducing inference latency, and deploying scalable cloud-native architectures for large-scale healthcare environments. Enhanced security mechanisms, federated learning approaches, and on-device inference pipelines can also be explored to strengthen privacy preservation and deployment flexibility [9, 10].

The proposed Ambient AI platform provides a strong foundation for future AI-driven healthcare documentation systems and can be further extended into a comprehensive intelligent clinical assistance ecosystem. ““

REFERENCES

- [1] Alec Radford, Jong Wook Kim, Tao Xu, and Greg Brockman. Robust speech recognition via large-scale weak supervision. *OpenAI Technical Report*, 2022.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, and Jakob Uszkoreit. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [3] Patrick Lewis, Ethan Perez, and Aleksandra Piktus. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] pgvector Contributors. pgvector: Open-source vector similarity search for postgresql. <https://github.com/pgvector/pgvector>, 2025.
- [5] Lawrence Weed. Medical records that guide and teach. *New England Journal of Medicine*, 278(11):593–600, 1968.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019.
- [8] Roger S. Pressman. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill, 8 edition, 2014.
- [9] Auth0. Json web token introduction. <https://jwt.io/introduction>, 2025.
- [10] Niels Provos and David Mazieres. bcrypt password hashing function. <https://github.com/pyca/bcrypt>, 2025.
- [11] Sebastian Ramírez. Fastapi framework documentation. <https://fastapi.tiangolo.com>, 2025.
- [12] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2025.
- [13] LangChain. Langchain framework documentation. <https://python.langchain.com>, 2025.

“‘latex id="n2r8xa"”

Appendix A

APPENDICES

A.1 Screen Shots

This section presents screenshots of the developed Ambient AI clinical documentation platform. The screenshots include clinician voice enrollment interfaces, dashboard interfaces, consultation processing outputs, SOAP note generation results, speaker diarization outputs, Hybrid RAG retrieval results, and patient history management interfaces. These screenshots demonstrate the successful implementation of frontend interfaces and backend integration workflows used within the proposed system.

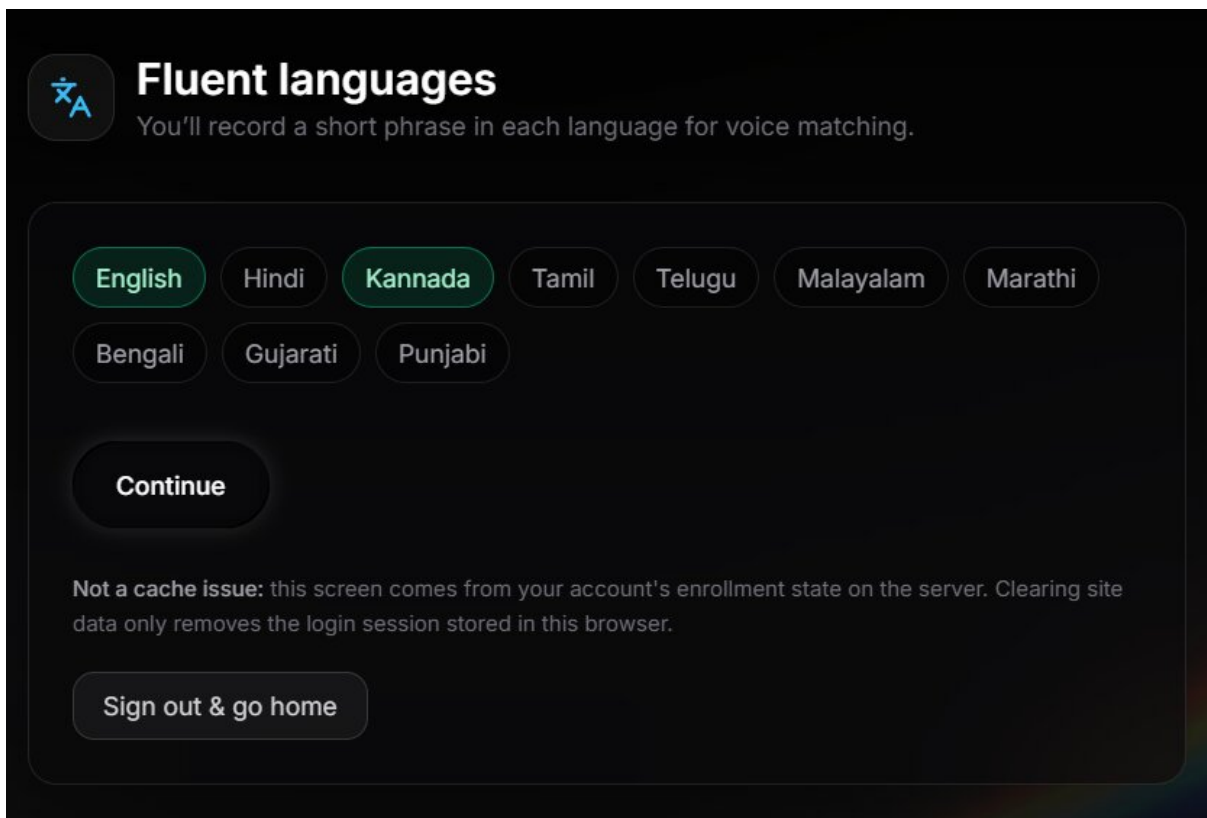


Figure A.1: Doctor Language Selection Interface

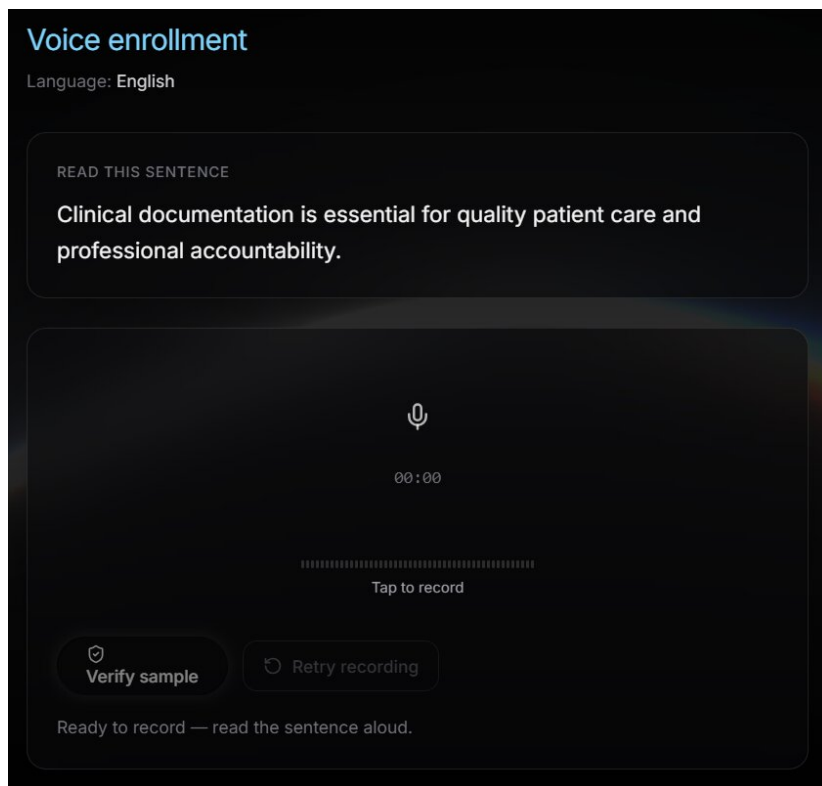


Figure A.2: Doctor Voice Enrollment Interface

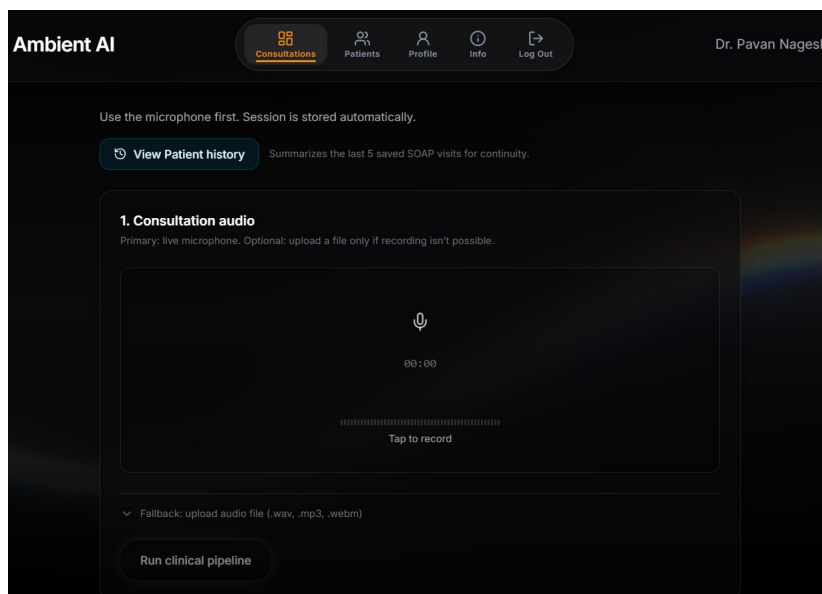


Figure A.3: Clinician Dashboard Interface

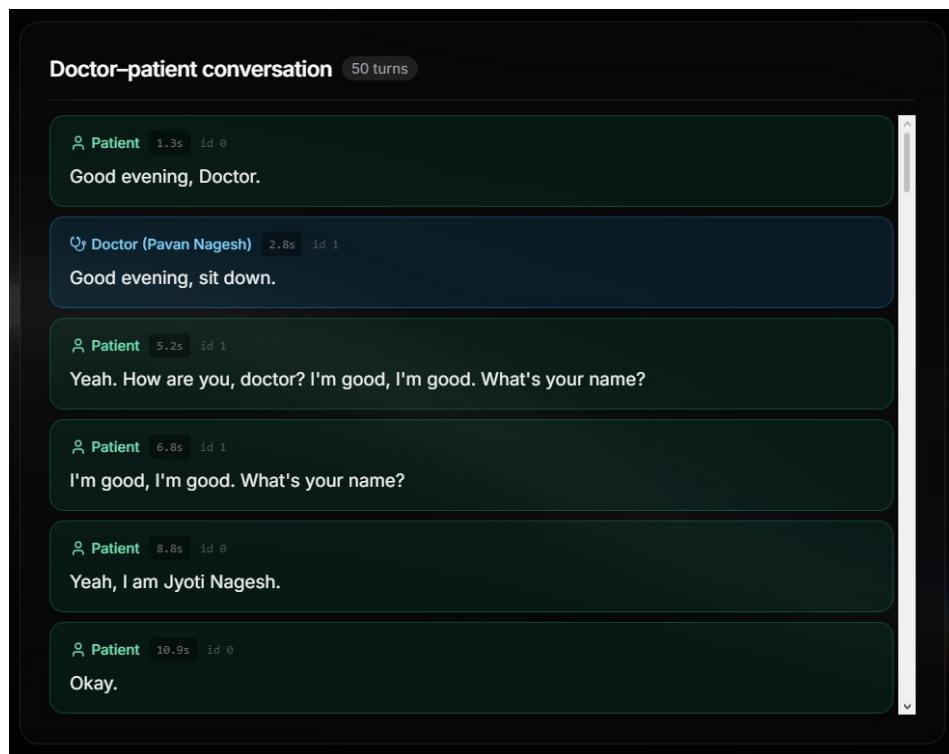


Figure A.4: Speaker Diarization Output

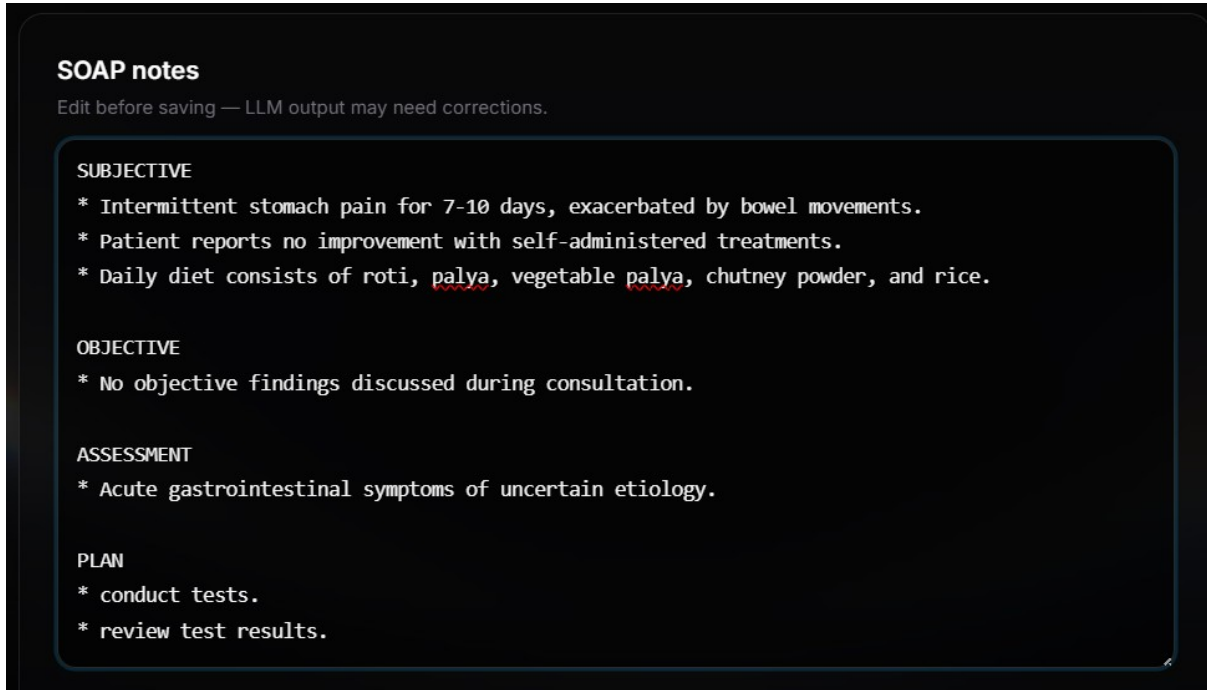
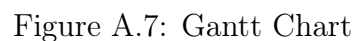


Figure A.5: Generated SOAP Notes

A.2 Gantt Chart



47

A.3 Glossary

- **ASR:** Automatic Speech Recognition used for converting consultation speech into text.
- **LLM:** Large Language Model used for semantic understanding and SOAP note generation.
- **SOAP Notes:** Structured clinical documentation containing Subjective, Objective, Assessment, and Plan sections.
- **RAG:** Retrieval-Augmented Generation used for continuity-aware consultation summarization.
- **PII:** Personally Identifiable Information requiring privacy-preserving redaction.
- **DAG:** Directed Acyclic Graph representing the AI workflow pipeline.
- **Embedding:** Semantic vector representation used for similarity-based retrieval.
- **Frontend:** User interface developed for clinician interaction and consultation management.
- **Backend API:** Server-side processing layer handling AI workflow execution.

A.4 Description of Tools and Technology Used

The proposed Ambient AI system was implemented using modern frontend, backend, database, and Artificial Intelligence technologies. The frontend application was developed using React.js, TypeScript, and Tailwind CSS for responsive user interface development. Backend APIs were implemented using FastAPI and Python for efficient AI pipeline execution.

PostgreSQL and pgvector were utilized for relational storage and semantic vector similarity retrieval operations. Automatic Speech Recognition and speaker diarization were implemented using AI speech processing frameworks, while Large Language Models were used for SOAP note generation and clinical entity extraction. The system additionally utilized Hybrid Retrieval-Augmented Generation for continuity-aware consultation summarization.

- **Frontend Technologies:** React.js, TypeScript, Tailwind CSS
- **Backend Technologies:** FastAPI, Python
- **Database Technologies:** PostgreSQL, pgvector
- **AI Technologies:** ASR, Speaker Diarization, Large Language Models

- **Retrieval Technologies:** Hybrid RAG, Semantic Vector Search
- **Deployment Utilities:** REST APIs, JSON-based communication

“\latex

A.5 Blue Print

The blueprint section contains the major workflow diagrams, architectural representations, database structures, and implementation flow diagrams used during development of the proposed Ambient AI clinical documentation platform.

The attached blueprint diagrams include:

- Overall System Architecture Diagram
- Clinical Consultation Workflow Diagram
- State Transition Diagram
- Sequence Diagram
- Entity Relationship Diagram
- Class Diagram
- Module Interaction Workflow
- Hybrid RAG Retrieval Workflow

These diagrams collectively represent the implementation methodology, AI workflow pipeline, database organization, and operational flow of the proposed system. The blueprint diagrams are attached at the end of the report for reference and evaluation purposes.

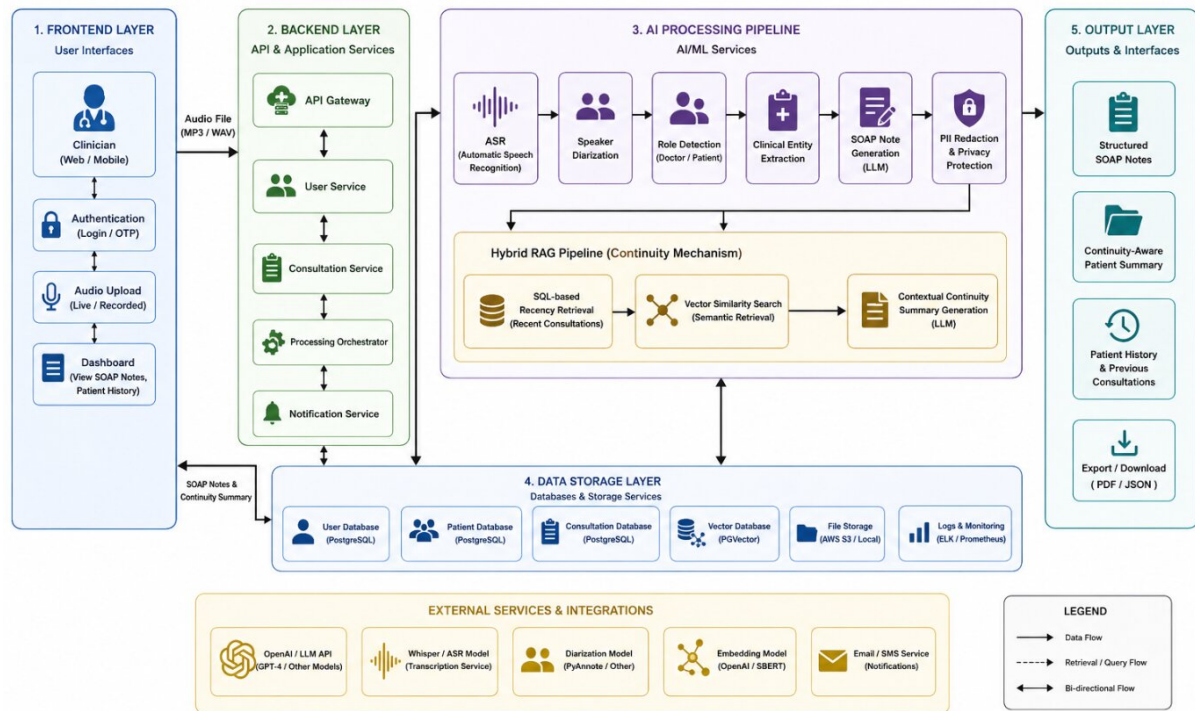


Figure A.8: Overall System Architecture

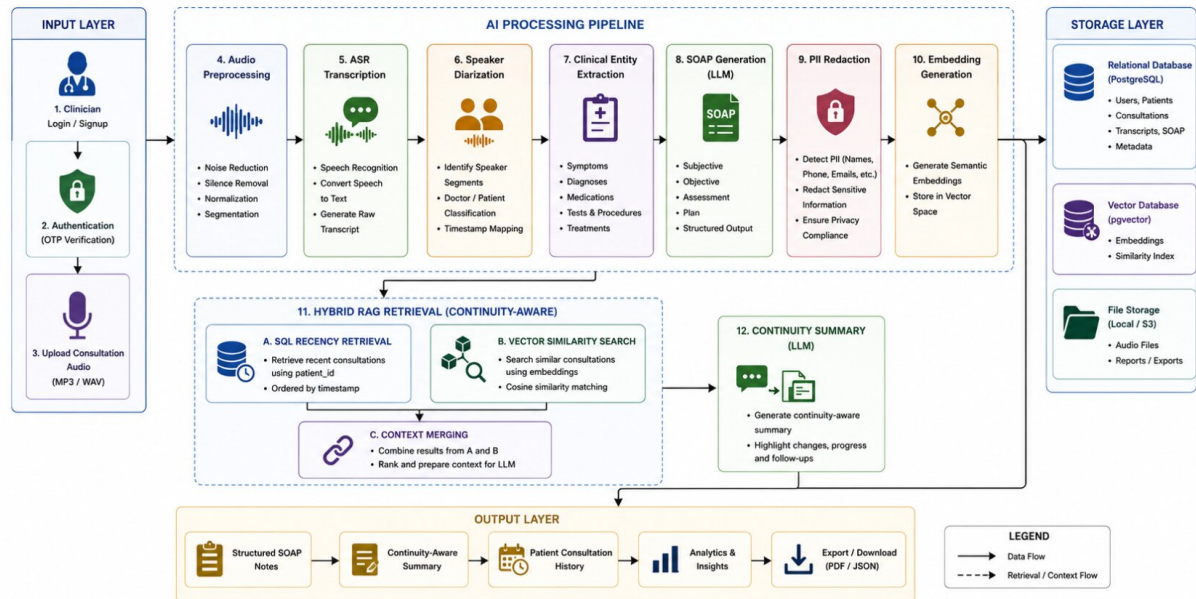


Figure A.9: Clinical Consultation Workflow Diagram

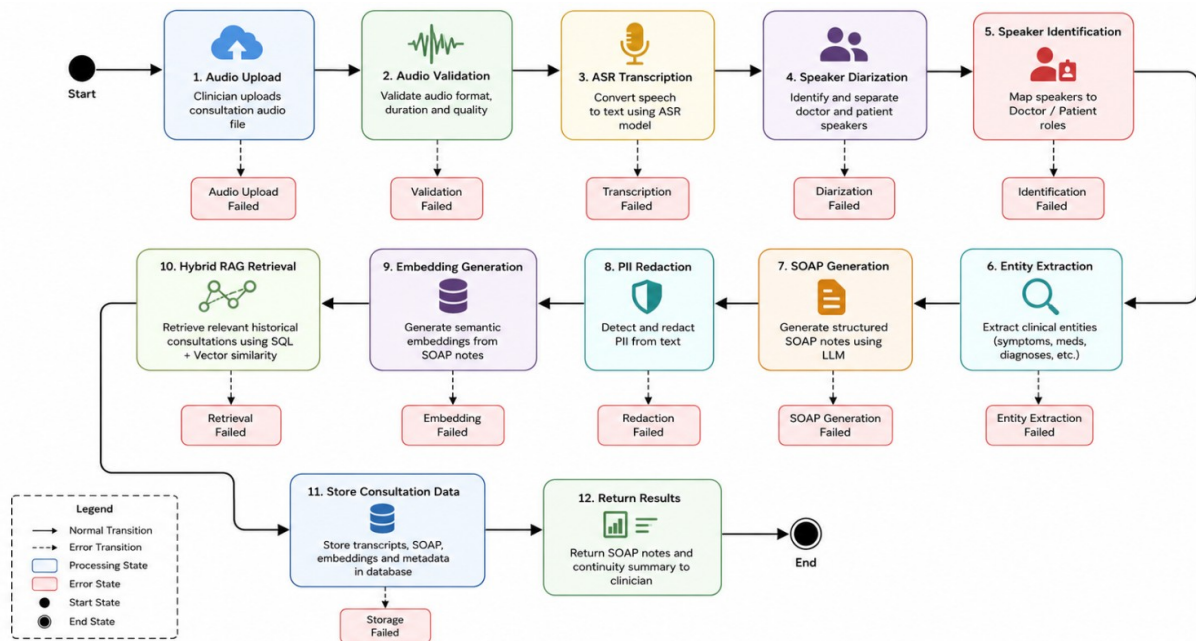


Figure A.10: State Transition Diagram

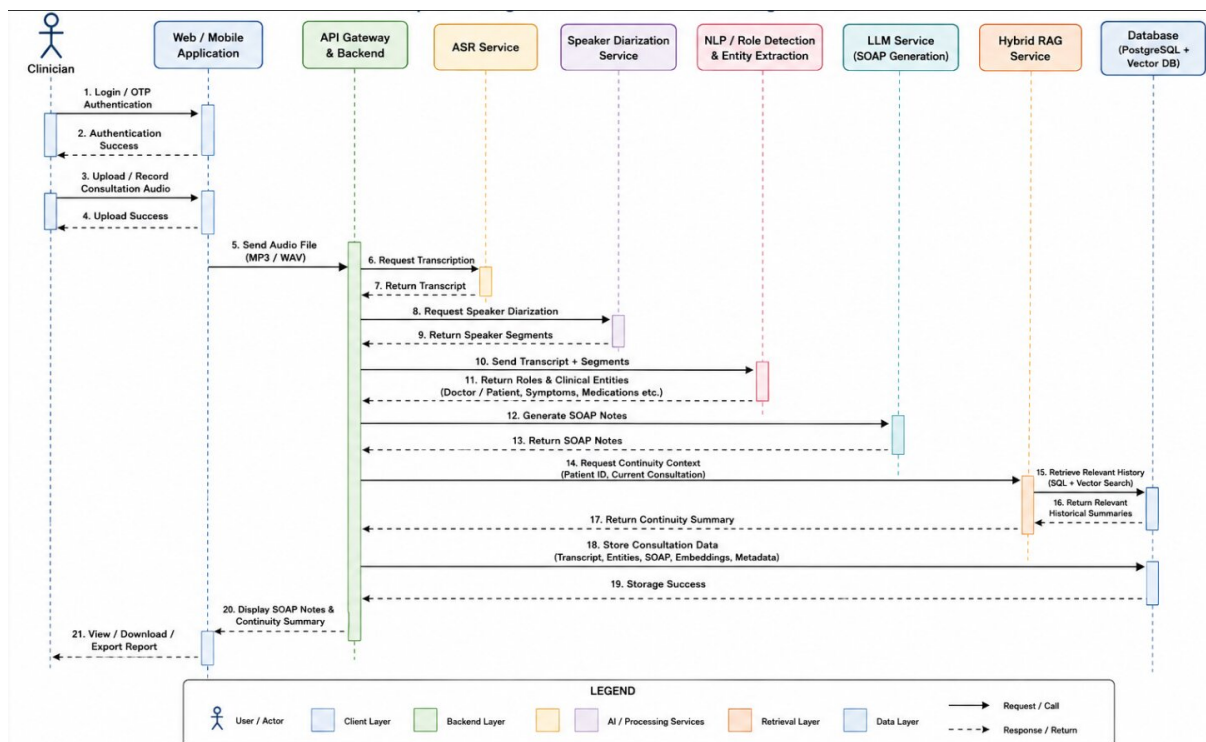


Figure A.11: Sequence Diagram

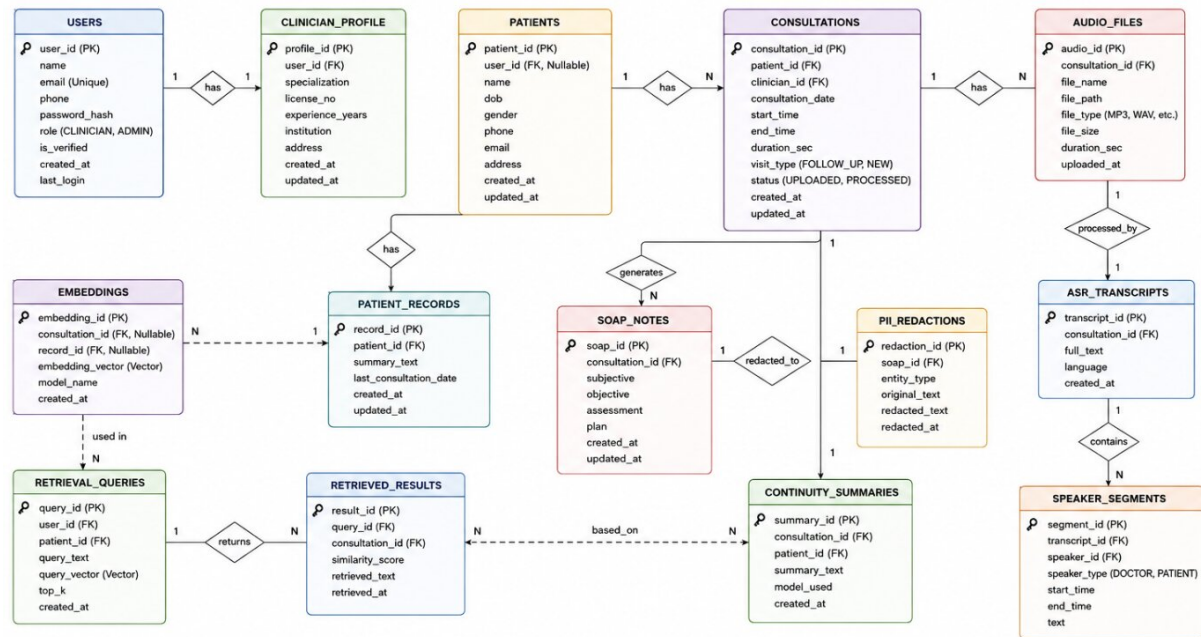


Figure A.12: Entity Relationship Diagram

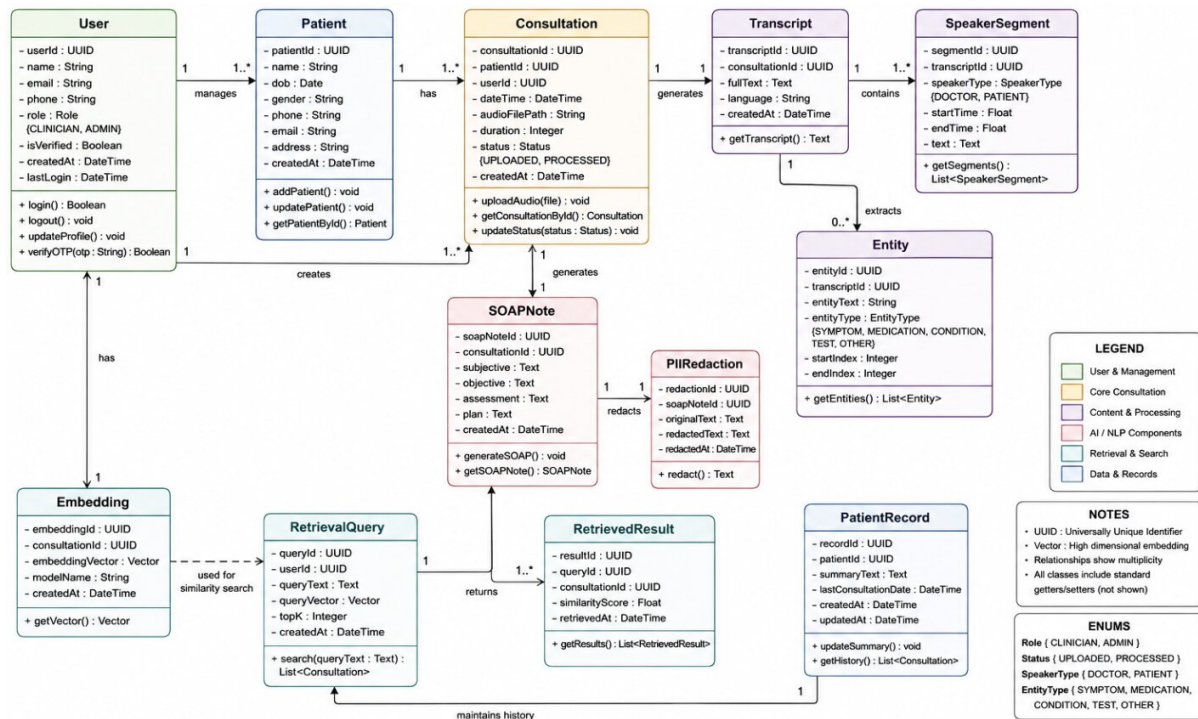


Figure A.13: Class Diagram

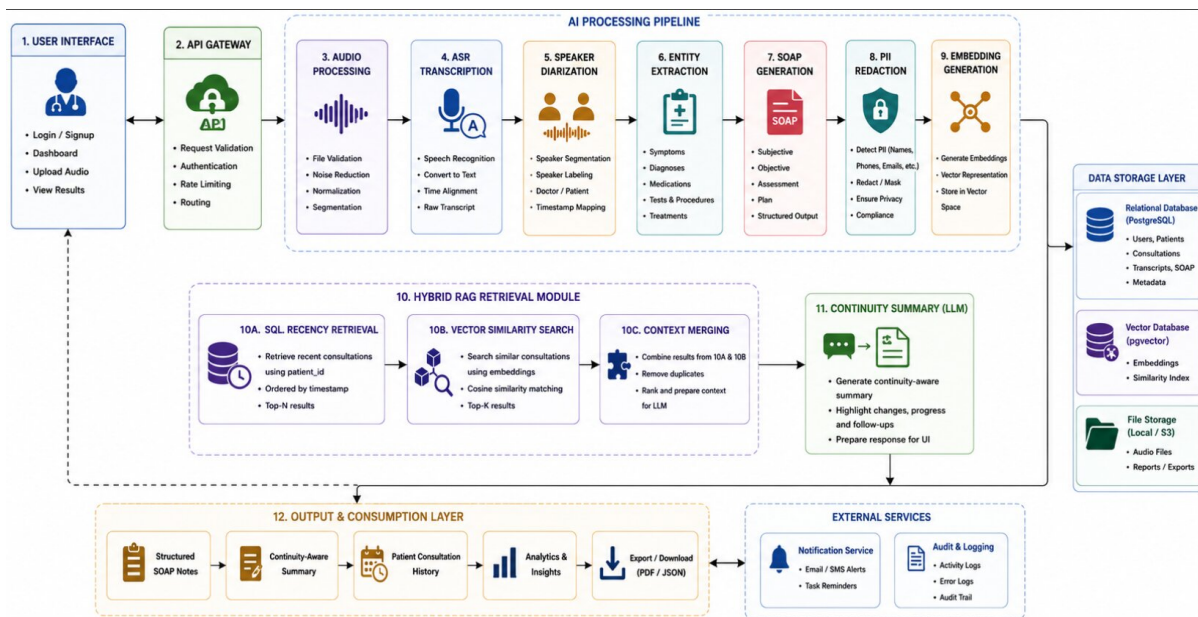


Figure A.14: Module Interaction Workflow

System Workflow of the Proposed Ambient AI Platform

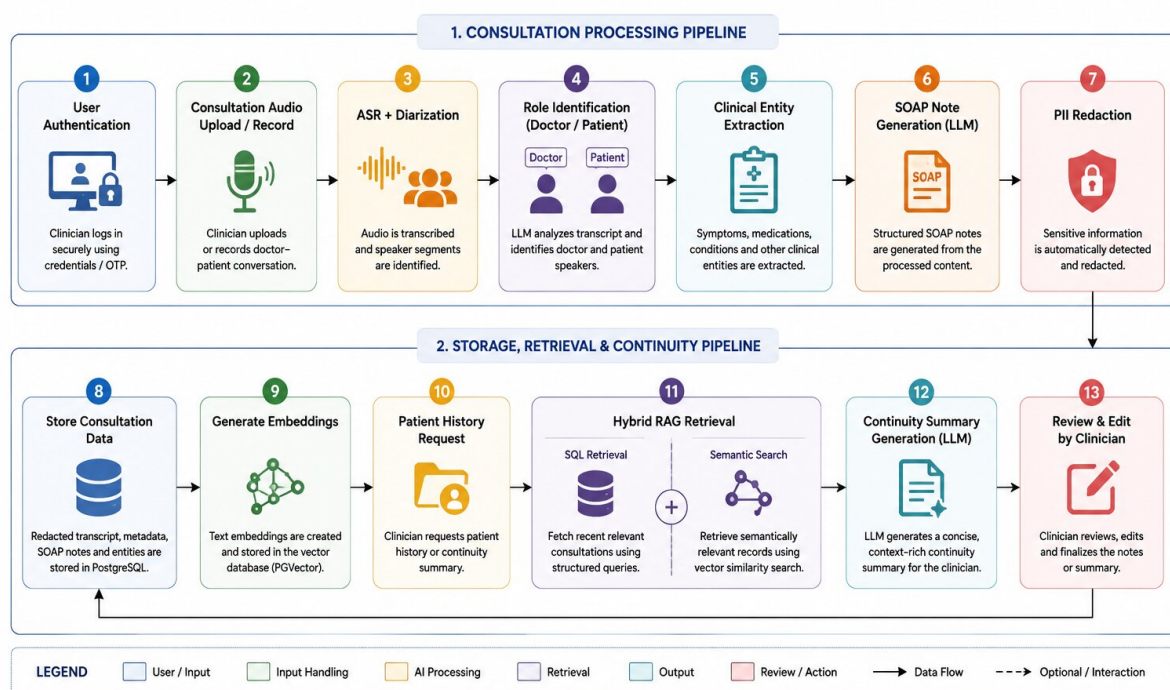


Figure A.15: Hybrid RAG Retrieval and System Workflow